Deep Learning-Based Intelligent Visual Diagnosis of Tomato Yellow Leaf Curl Virus Using Leaf Images: A Leakage-Aware Framework, Data Audit and Decision-Support Prototype

Ritesh Dwivedi[1], Sneha Garg[1], Navneet Kaur[1]

[1] Department of Computer Science and Engineering, Chandigarh University, Gharuan, Punjab, India

Corresponding author: Ritesh Dwivedi (25mds10023@cuchd.in)

Abstract

Tomato yellow leaf curl virus (TYLCV) causes severe losses in tomato, and rapid, correct field screening would support timely vector control and removal of infected plants. Deep learning can recognise TYLCV-like symptoms in leaf photographs, but published accuracies are usually measured on lab-background benchmarks with random image-level splits, and little is known about field behaviour. This paper presents a leakage-aware framework for leaf-image TYLCV screening and a decision-support prototype built around it. The framework combines duplicate-aware data splitting, transfer-learning classifiers with temperature-scaled probabilities and Grad-CAM explanations, a staged evaluation protocol organised as a six-level maturity ladder, and modules that connect a screening result to risk, resistance, insecticide-rotation and surveillance information. We report four sets of results. First, an audit of a public TYLCV image folder (100 photographs) shows that every image is positive, that 91% are lab-style images, that leaf coverage alone separates lab-style from field photographs without overlap, and that two photographs occur twice at different resolutions. Second, a controlled experiment on synthetic images quantifies how a random split inflates accuracy relative to a duplicate-aware split. Third, on a simulated benchmark a calibrated model predicts outbreaks better than heuristic and single-factor baselines (ROC-AUC 0.98 versus 0.93 and 0.64–0.76), with bootstrap intervals. Fourth, the prototype passes 34 automated tests. No image classifier has yet been trained on real labelled leaf photographs, because no public set with negatives and field images was available to us; we therefore make no claim about field diagnostic accuracy, and we specify the data and protocol needed to establish it.

Keywords: tomato yellow leaf curl virus; deep learning; leaf-image diagnosis; data leakage; calibration; explainable AI; domain shift; decision support; precision agriculture

1. Introduction

Tomato (*Solanum lycopersicum*) is one of the most widely grown vegetable crops, and its production is constrained by many diseases. One of the most damaging is tomato yellow leaf curl disease, caused by tomato yellow leaf curl virus (TYLCV). Infected plants develop upward-curling, yellowing leaves and stunted growth, and when plants are infected early almost no marketable fruit is produced [1]. The virus is transmitted by the whitefly *Bemisia tabaci* in a persistent manner [2], can be seed-transmissible [3], and has spread across the world through trade and the movement of its vector [4]. Because no cure exists for an infected plant, control relies on keeping whiteflies away, growing resistant cultivars and removing infected plants early [1], [5], [6].

Early removal depends on early detection, and in practice detection is done by people who recognise the symptoms by eye. That skill is scarce and subjective at the scale of commercial production, and many smallholders have no access to it. Smartphones, in contrast, are common, which has motivated a large body of work on diagnosing plant diseases from leaf photographs with deep learning [7], [8], [9], [10], [11]. Reported accuracies on the PlantVillage benchmark exceed 99% [7], [8], yet the first large study already reported that accuracy fell sharply on images taken under other conditions [7], and later work shows that benchmark backgrounds can leak the class label [12], [13] and that field performance is considerably lower than benchmark performance [14], [15], [16].

Three practical problems follow for TYLCV. The first is evidence: we are not aware of a public TYLCV leaf-image collection that contains healthy leaves, look-alike disorders, field photographs and molecular confirmation. The second is evaluation: when the same photograph appears more than once, or when lab and field images are mixed, a random image-level split gives optimistic scores. The third is use: even a good classifier answers only one question (*does this leaf look infected?*), whereas a grower needs to decide what to do next, which depends on vector pressure, the resistance of the cultivar and the prevalence of the virus [17], [18].

1.1 Contributions

This paper addresses these problems with a framework, an implementation and four sets of experiments. Its contributions are:

- **A leakage-aware framework** for leaf-image TYLCV screening: duplicate-aware group splitting, transfer-learning classifiers, temperature-scaled probabilities with a validation-chosen threshold, Grad-CAM explanations, and an evaluation protocol organised as a six-level maturity ladder (Section 4).
- **A decision-support prototype, TYLCV-Guard**, implemented with the Python standard library, which connects a screening result to a climate and vector risk index, a resistance-stacking planner, an insecticide-rotation planner, a recombinant-screening tool, a seed-lot register, a field-trial log with a calibrated model, and an optional hosted image model (Section 5).
- **An audit of a public TYLCV image folder** with real photographs, showing which properties make it unsuitable for estimating diagnostic accuracy and how large the lab-to-field gap is in simple image statistics (Section 7.1).
- **A controlled leakage experiment** on synthetic images that quantifies how much a random split inflates accuracy relative to a duplicate-aware split (Section 7.2).
- **A simulated benchmark with sensitivity analyses** that compares previous single-factor approaches, a heuristic model and a calibrated model, with bootstrap confidence

intervals, noise and training-size sensitivity and cost-based threshold analysis (Section 7.3).

1.2 Summary of findings and a note on scope

On the simulated benchmark the calibrated model reached a ROC-AUC of 0.980 against 0.930 for the heuristic model and 0.635 to 0.764 for single-factor baselines; the difference between the calibrated and heuristic model was 0.050 (95% bootstrap interval 0.035 to 0.067). In the audit of the public folder, leaf coverage differed by roughly an order of magnitude between lab-style and field photographs. These results describe how the components behave under stated assumptions and in a specific dataset; they are not estimates of diagnostic accuracy on farms. In particular, **no image classifier was trained or evaluated on real labelled leaf photographs**, because the only real TYLCV photographs available to us were all positive and no labelled negatives or pretrained weights could be obtained in our environment. Section 9 discusses this limitation, and Section 11 states what is needed to remove it.

1.3 Organisation of the paper

Section 2 reviews related work. Section 3 formulates the problem. Section 4 describes the framework and Section 5 its implementation. Section 6 describes the experimental design and Section 7 reports the results. Sections 8 to 10 discuss the findings, limitations and ethical considerations, and Section 11 concludes. Appendices give the API reference, configuration, reproduction instructions and the test inventory.

2. Related Work

2.1 Deep learning for plant disease diagnosis

Early plant-disease studies fine-tuned general-purpose CNNs on leaf photographs [19], [7]. The PlantVillage benchmark, with tens of thousands of lab-background leaf images across many crop–disease classes, became the standard test bed, and models reached accuracies above 99% [7], [8]. Comparative studies found that several mainstream networks reach similar accuracy on it [20]. Tomato-specific work compared standard CNNs [21], proposed lightweight networks [22] and added attention modules to residual networks [23], [24]. Purpose-built CNNs and other crops tell a similar story of high lab accuracy [25]. Surveys describe the architectures, datasets and open problems [9], [10], [11], [26].

2.2 Generalisation, dataset bias and leakage

The gap between laboratory and field is the central finding of the literature. A network that reached very high accuracy on held-out PlantVillage images dropped to roughly 31% on images from other conditions [7]. Studies of dataset size and variety conclude that diversity of capture conditions matters more than additional images of the same kind [14], and that classifying individual lesions can be more robust than whole leaves [27]. Work on smartphone

images in the wild shows that training with field-captured images is needed [28], [29], and one study argues for training that reflects practical variation [16]. PlantVillage backgrounds alone carry enough information to classify images well above chance [12], an example of shortcut learning [13], and the PlantDoc dataset was assembled to provide harder in-the-wild images [30]. Work that addresses the current limitations of plant-disease detection stresses dataset size, variety and real-world capture [15]. Distribution-shift benchmarks in general machine learning show that models that look excellent in-distribution can fail under shift [31].

2.3 Detection, deployment, calibration and explanation

When several symptoms appear in one image, detectors are more informative than classifiers; real-time detection of tomato diseases and pests has been demonstrated on field images [32], and greenhouse systems have combined detection with severity estimation [33]. Field use favours small models [34], [35]. Modern networks tend to be over-confident, and temperature scaling is a simple remedy [36]; uncertainty can also be estimated with ensembles [37], although uncertainty degrades under shift [38]. Grad-CAM highlights the regions that drove a decision [39] and has been used for tomato classifiers [21], but saliency methods need validation [40].

2.4 TYLCV management context

An image result is useful only if it leads to the right action. Resistance genes (*Ty-1* to *Ty-6*) from wild tomato relatives are widely used but not fully durable [5], [18], [41]; recombination and spread continue [4], [42]; related viruses such as tomato leaf curl New Delhi virus show how other lineages can emerge [43]; whitefly control is complicated by insecticide resistance [17], [44] and by climate change [45]. Integrated programmes combine exclusion, resistant cultivars, vector monitoring and early removal [6]. These facts motivate the decision-support modules of Section 5.

2.5 Summary and positioning

1 summarises key studies and 2 states what the present paper adds. The studies differ in data and protocol and are not ranked. The gap in the literature is not another architecture but a combination of evidence, evaluation and decision context for a specific, economically important virus.

Table 1. Summary of key related studies

Ref.	Year	Focus	Main point for TYLCV screening
[19]	2016	CNN leaf-disease recognition	Early demonstration with web-collected leaf images
[7]	2016	PlantVillage classification	99.35% on held-out lab images; about 31% on other-condition images
[21]	2017	Tomato diseases, saliency	Visualisation localises symptoms
[29]	2017	Cassava, field images	Feasibility with images from real fields
[32]	2017	Tomato detection	Real-time detection of diseases and pests in situ
[8]	2018	Diagnosis, 58 classes	99.53% success on an open database
[14]	2018	Dataset size and variety	Variety of conditions matters most
[20]	2019	Fine-tuning comparison	Similar benchmark accuracy across networks
[28]	2019	Smartphone field images	Field-captured training data needed
[15]	2019	Limits of current approaches	Real-world capture is the main obstacle
[30]	2020	In-the-wild dataset	Harder test conditions
[33]	2020	Greenhouse tomato	Detection plus severity estimation
[16]	2020	Characterising disease	Train for practical variation
[12]	2022	Dataset bias	PlantVillage backgrounds carry class information

Table 2. What this paper adds to the typical benchmark-style approach

Aspect	Typical benchmark-style study	This paper
Splitting	Random image-level split	Near-duplicate-aware group split, with a leakage experiment
Data checking	Rarely reported	Audit of a public TYLCV folder: sources, duplicates, label coverage
Outputs	Class label and accuracy	Calibrated probability, validation-chosen threshold, heat-map
Evaluation	Held-out images of the same kind	Maturity ladder: external field set, hard negatives, shortcut checks, molecular subset
Context	Image only	Risk index, resistance planner, rotation planner, surveillance, seed lots
Claims	Accuracy on benchmark	Explicit separation of real, synthetic and simulated evidence

3. Problem Formulation

3.1 Screening task and notation

Let x denote an RGB photograph of a tomato leaf, leaflet or plant part, and let $y \in \{0, 1\}$ denote its true state, with $y = 1$ meaning TYLCV infection confirmed by a molecular test and $y = 0$ meaning not infected (healthy or affected by another condition). A screening model with parameters θ maps x to logits $z(x; \theta) \in \mathbb{R}^2$, which are converted to a probability $\hat{p}(x) = P(y = 1 \mid x)$ by the softmax function with a temperature T:

$$\hat{p}(x) = \exp(z_1 / T) / [\, \exp(z_0 / T) + \exp(z_1 / T) \,] \quad (1)$$

A decision rule flags the image when $\hat{p}(x) \geq \tau$, where τ is a threshold. The model is trained on a training set, the temperature T and the threshold τ are fitted on a validation set, and

performance is measured on a test set. The training objective is the class-weighted cross-entropy:

$$L(\theta) = -(1/N) \Sigma_i \, w(y_i) \, [\, y_i \log \hat{p}(x_i) + (1 - y_i) \log(1 - \hat{p}(x_i)) \,] \qquad (2)$$

3.2 Splits and leakage

Two images are near-duplicates if they show the same photograph, for example re-encoded at a different resolution or with a slightly different brightness. Let $g(x)$ be a group identifier such that near-duplicates share a group. A split is **leakage-free** if no group appears in more than one of the training, validation and test sets. A random image-level split is not leakage-free when groups contain more than one image. The *leak fraction* of a split is the share of test images whose group also appears in the training set:

$$\text{leak} = (1 / |\text{Test}|) \, \Sigma_\{x \in \text{Test}\} \, 1[\, g(x) \in g(\text{Train}) \,] \qquad (3)$$

If a model can memorise individual images, a high leak fraction inflates test accuracy; the experiment in Section 7.2 measures how much.

3.3 Calibration, thresholds and the cost of errors

The temperature is chosen to minimise the negative log-likelihood on validation data, and calibration is summarised by the expected calibration error over probability bins:

$$T^\star = \arg\min_T \, \text{NLL_val}(z / T), \qquad \text{ECE} = \Sigma_m \, (|B_m| / N) \cdot |\, \text{acc}(B_m) - \text{conf}(B_m) \,| \qquad (4)$$

The threshold can be chosen to maximise F1 on validation data, or to minimise expected cost when a missed infection (false negative, FN) and a false alarm (false positive, FP) have different costs c_FN and c_FP:

$$\tau^\star = \arg\min_\tau \, [\, c_FN \cdot FN(\tau) + c_FP \cdot FP(\tau) \,] / N \qquad (5)$$

The usefulness of a flagged result depends on how common infection is. With sensitivity *Se*, specificity *Sp* and prevalence π, the positive predictive value is

$$PPV = Se \cdot \pi / [\, Se \cdot \pi + (1 - Sp) \cdot (1 - \pi) \,] \qquad (6)$$

For example (hypothetical numbers), with *Se* = *Sp* = 0.95 the PPV is 50% at π = 5% and about 83% at π = 20%. A screening result should therefore be read together with local prevalence, which the decision-support modules help to estimate.

3.4 Decision-support tasks

Beyond the classifier, four decision problems arise in TYLCV management, and the prototype addresses each with a module: (i) estimating the *infection risk* of a field from temperature, vector density, crop stage, resistance and exclusion; (ii) comparing *resistance stacks* (combinations of *Ty* loci) in terms of expected protection and durability; (iii) planning *insecticide rotation* so that the same mode of action is not used consecutively and failed groups

are excluded; and (iv) *surveillance* of virus sequences, including screening for recombinants. Each module is advisory and rests on stated assumptions (Section 4.7).

3.5 Research questions

The study is organised around five research questions:

- **RQ1.** What do the available real TYLCV photographs allow us to conclude, and what do they not allow? (Section 7.1)
- **RQ2.** How much can a random split inflate measured accuracy relative to a duplicate-aware split when near-duplicates exist? (Section 7.2)
- **RQ3.** Under simulated conditions, how do a heuristic model and a calibrated model compare with single-factor baselines, and how certain are the differences? (Section 7.3)
- **RQ4.** How sensitive are these conclusions to observation noise, training-set size and the choice of decision threshold? (Section 7.3)
- **RQ5.** Can the framework and prototype be implemented, tested and reproduced with modest resources? (Sections 5 and 7.4)

4. Proposed Framework

4.1 Overview

The framework has five steps (1). Leaf photographs are collected from several sources (Step 1) and pre-processed, with near-duplicate images grouped before any split is made (Step 2). A transfer-learning classifier is trained (Step 3), its outputs are calibrated and explained (Step 4), and the result is evaluated under a staged protocol (Step 5). A decision-support layer (the purple box in the figure) supplies context for the final decision: risk, resistance, vector management and surveillance. The same schematic is used throughout the paper to indicate which step each result belongs to.

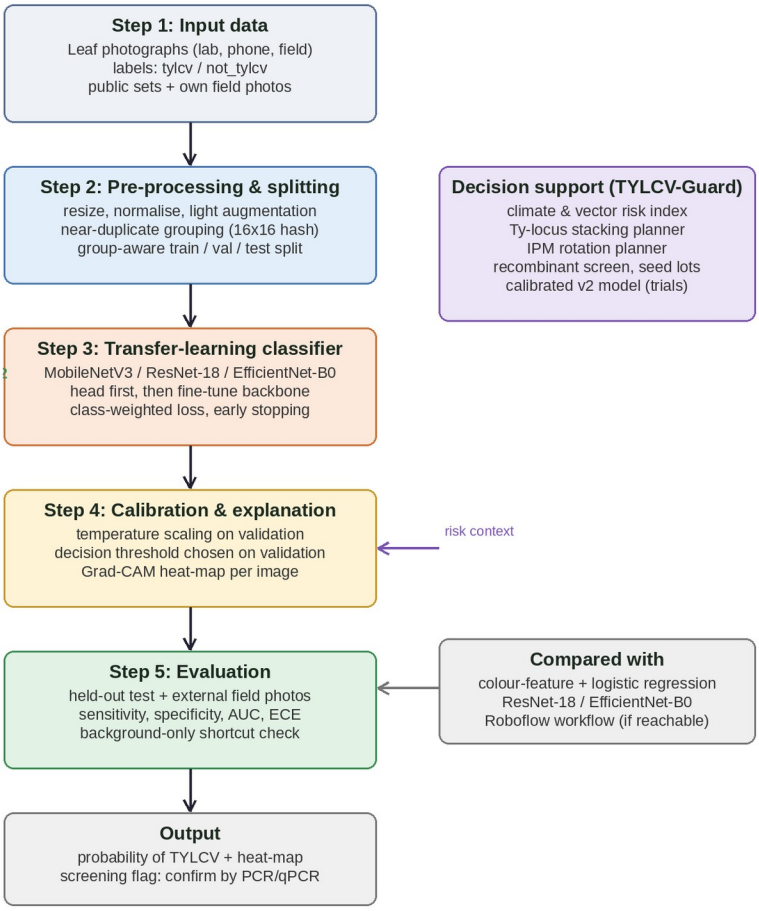


Figure 1. Proposed framework: what happens to the data at each step and where decision support is used.

4.2 Data and duplicate-aware splitting

Splitting is the step most often done carelessly and the one that most affects reported accuracy. Our procedure computes a 16 × 16 average hash of each image (256 bits), links every pair of images whose hashes differ in at most 12 bits using union–find, and treats each connected component as a group. Groups are then assigned, class by class, to the training (70%), validation (15%) and test (15%) sets so that no group is divided (2). The 12-bit threshold was chosen by inspecting the audited folder of Section 7.1, where it linked the two true duplicate pairs and no unrelated images. A coarser 8 × 8 hash was tried first and was rejected, because it merged unrelated leaves photographed on plain backgrounds into very large groups and left the validation split empty in a test; this is the kind of failure that a group-aware procedure must be checked against.

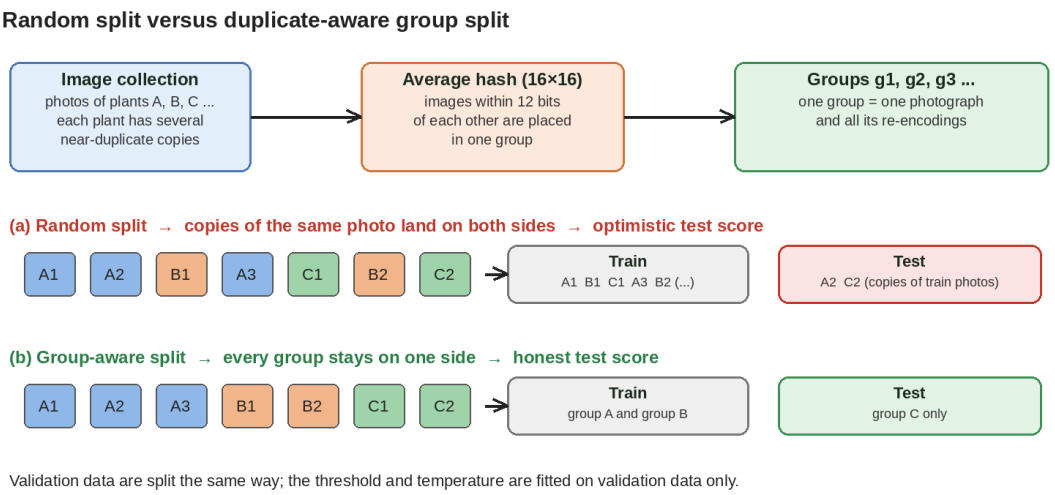


Figure 2. Random split compared with duplicate-aware group split. Copies of the same photograph (A1, A2, A3) are kept together in the group-aware split.

The procedure has limitations. Hashing detects re-encodings, rescaling and small brightness changes, but not different photographs of the same plant, which must be grouped using metadata (plant identifier, capture session) when it exists. A group that contains a large fraction of the dataset makes balanced splitting impossible, and the implementation warns when one group exceeds 20% of the images. Our leakage experiment (Section 7.2) also shows that residual leakage remains when variants differ more than the threshold allows.

4.3 Transfer-learning classifier

The classifier is a pretrained convolutional network with its final layer replaced by a two-class linear head (3). MobileNetV3-Small is the default because it is small enough for phones [34]; ResNet-18 [46] and EfficientNet-B0 [47] are available as larger baselines. Vision transformers [48] are a possible extension, but they are data-hungry and the small datasets typical of plant disease favour convolutional backbones. Training proceeds in two phases: only the new head is trained for the first two epochs, and then the whole network is fine-tuned at a ten-times lower learning rate. The loss is the class-weighted cross-entropy of Equation 2, the optimiser is AdamW, and training stops early when validation AUC has not improved for four epochs. 3 lists the configuration.

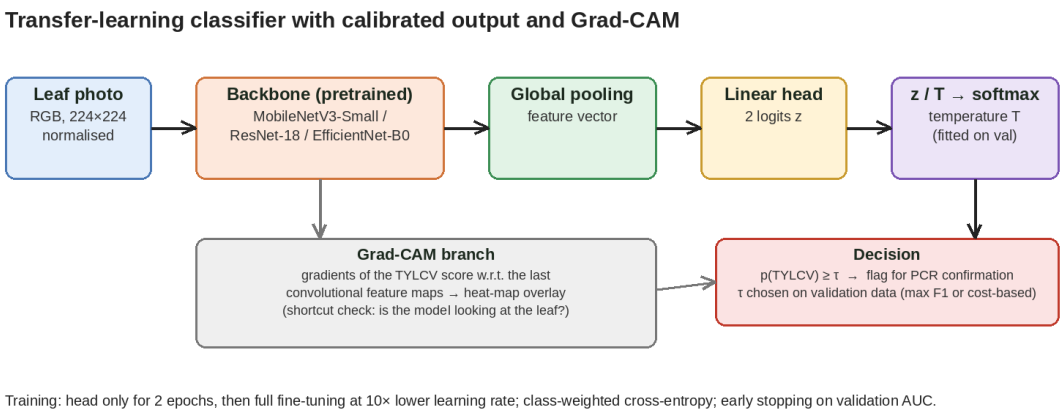


Figure 3. Transfer-learning classifier with temperature-scaled output and a Grad-CAM branch.

Table 3. Classifier and training configuration

Item	Setting
Backbone	MobileNetV3-Small (default), ResNet-18, EfficientNet-B0; ImageNet-pretrained when weights are available
Input	224 × 224 RGB; resize to 1.14× then centre-crop; ImageNet mean and standard deviation
Augmentation (training only)	Random resized crop (scale 0.6–1.0), horizontal and vertical flips, rotation up to 25°, brightness 0.25, contrast 0.25, saturation 0.2, hue 0.03
Head	Final linear layer replaced by 2 outputs (not_tylcv, tylcv)
Loss	Class-weighted cross-entropy; weights inversely proportional to class frequency
Optimiser	AdamW, weight decay 1e-4; learning rate 1e-3 for the head phase, 1e-4 for full fine-tuning
Schedule	2 frozen-backbone epochs; early stopping on validation AUC, patience 4
Fallback	If pretrained weights cannot be downloaded, a warning is printed and training starts from random initialisation

The augmentation keeps hue changes very small (0.03) because yellowing is the diagnostic cue for TYLCV and strong colour jitter could remove it. This is a design decision rather than a result of this study, and it should be tested once real data are available.

4.4 Calibration, threshold and explanation

After training, the temperature T is fitted on validation logits by L-BFGS, minimising the negative log-likelihood (Equation 5). The decision threshold τ is then chosen on validation data by a grid search over 0.05 to 0.95 that maximises F1, or by the cost-based rule of Equation 6 when error costs are specified. The test set is not used for either step. Explanations are generated by Grad-CAM [39]: the gradient of the TYLCV score with respect to the last convolutional feature maps is averaged to obtain channel weights, the weighted maps are summed and passed through a ReLU, and the result is upsampled to the image size. The target layer is `features.12` for MobileNetV3-Small, `layer4` for ResNet-18 and `features.8` for EfficientNet-B0. Because saliency maps can be misleading [40], they are used only as one of several shortcut checks (Section 4.5).

4.5 Evaluation protocol and maturity ladder

The evaluation protocol (4) separates what each test establishes. The internal test set, free of near-duplicate groups shared with training, measures in-distribution performance. An external field set from other sites, devices and backgrounds measures robustness to shift [31]. A hard-negative set measures false-positive rate on look-alike disorders. Calibration is evaluated with the ECE and reliability diagrams, and shortcut checks combine Grad-CAM with a background-only baseline that is trained on image borders only [12]. A subset with PCR or qPCR confirmation provides molecular ground truth.

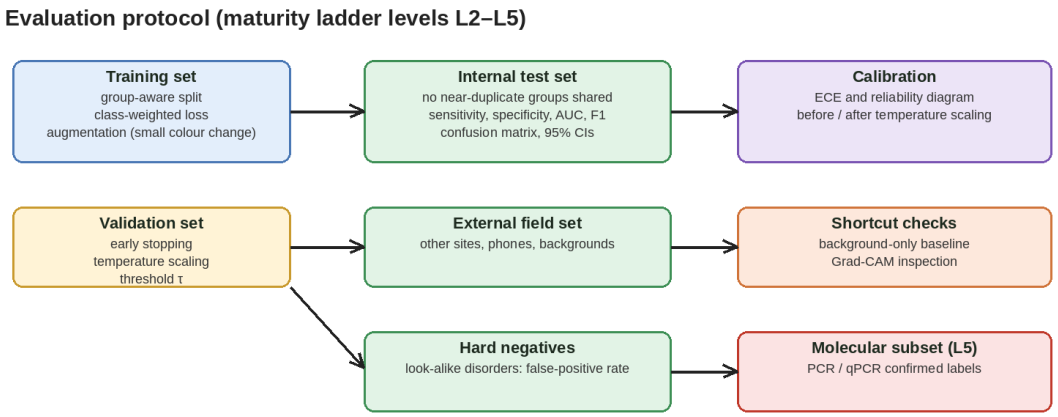


Figure 4. Evaluation protocol: which test establishes which claim.

We organise these tests as a six-level maturity ladder (5, 4). Levels are cumulative: a claim at level L4 presupposes the evidence of L2 and L3. Most published studies sit at L1 [7], [8], [20], and the work in this paper is at the level of design and tooling for L2–L5, not yet at their evidence for the image model.

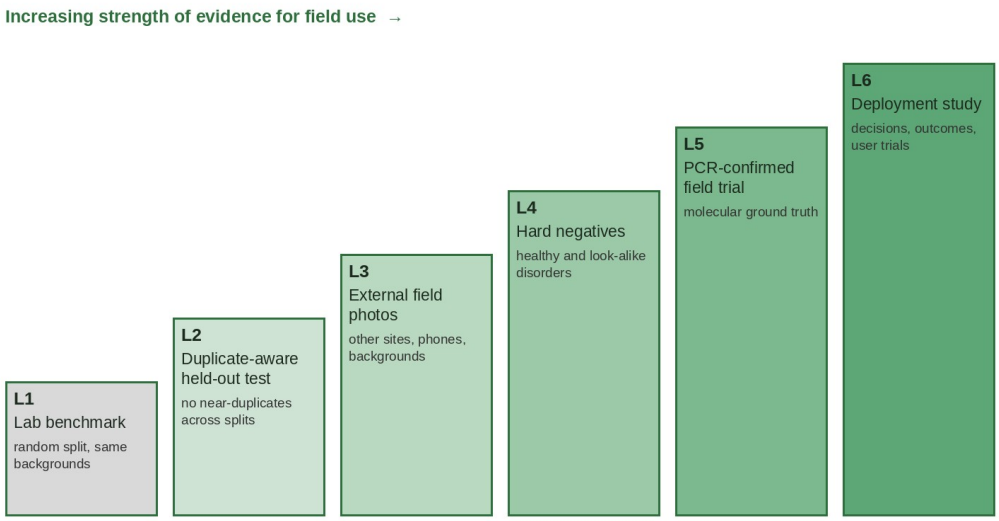


Figure 5. Evaluation maturity ladder for TYLCV leaf-image diagnosis.

Table 4. Maturity ladder: requirements and what each level establishes

Level	Requirement	What it establishes
L1	Lab benchmark, random split	Model capacity on benchmark-like data
L2	Duplicate-aware held-out test	In-distribution performance without leakage
L3	External field photographs	Robustness to new sites, devices and backgrounds
L4	Hard negatives (healthy and look-alike disorders)	Specificity against confusing conditions
L5	PCR/qPCR-confirmed field trial	Agreement with molecular ground truth
L6	Deployment study with user decisions	Effect on management outcomes

4.6 Confidence intervals

Proportions such as sensitivity, specificity and incidence are reported with Wilson 95% intervals. For k successes in n trials with $z = 1.96$, the interval is

$$[\ (\hat{p} + z^2/2n \pm z\ \sqrt{(\ \hat{p}(1-\hat{p})/n + z^2/4n^2\ )})\ /\ (1 + z^2/n)\ ],\qquad \hat{p} = k / n \tag{7}$$

Differences between models on the same test set are summarised by non-parametric bootstrap resampling of the test set (1,000 resamples), reporting the mean and the 2.5th to 97.5th percentiles.

4.7 Decision-support models

The decision-support layer is made of small, transparent models. All of them are heuristic or learned from user-supplied data, and all are advisory; none has been validated against field data.

4.7.1 Climate and vector risk index (v1)

The risk index combines four factors (6). The temperature factor f_T is piecewise linear through the control points of 5; the vector factor saturates with the logarithm of adult whitefly density w (adults per leaf), $f_V = \min(1, \ln(1 + w) / \ln 6)$; the stage factor f_S takes the values of 5; and resistance and exclusion reduce the index:

$$R = 100 \cdot f_T \cdot f_V \cdot f_S \cdot (1 - 0.7\ p) \cdot (0.25 \text{ if exclusion else } 1) \tag{8}$$

where $p \in [0, 1]$ is the protection score of the stacking planner (Section 4.7.2). The index is mapped to the categories Low (< 20), Moderate (< 40), High (< 65) and Very high. The curves encode qualitative statements from the literature: warm temperatures favour whitefly activity and virus spread [45], young plants are the most vulnerable [1], and exclusion and resistance reduce infection [5], [6]. They are priors, not fitted quantities.

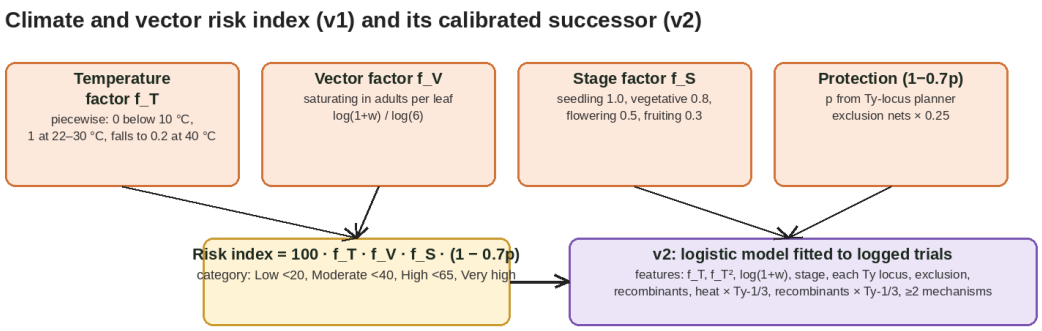


Figure 6. Computation of the climate and vector risk index (v1) and its calibrated successor (v2).

Table 5. Control points of the temperature factor (linear between points; 0 below 10 °C and 0.1 above 40 °C) and stage factors

Temperature (°C)	**f_T**
10	0
18	0.6
22	1
30	1
35	0.6
40	0.2
Crop stage	f_S
seedling	1
vegetative	0.8
flowering	0.5
fruiting	0.3

4.7.2 Resistance stacking planner

Each *Ty* locus is given an illustrative effect weight *w* and a mechanism (6). Effective weights are reduced for *Ty-1/Ty-3* when the mean maximum temperature exceeds 30 °C (×0.7) and when recombinant viruses are present (×0.6), and for *Ty-2* against begomoviruses other than TYLCV (×0.5), reflecting reported limits of these loci [5], [18]. The combined protection of a stack is

$$p = \min(0.95, 1 - \Pi_j (1 - w_{j,\mathrm{eff}})) \qquad (9)$$

Durability is classified by the number of distinct mechanisms among loci with weight ≥ 0.2: one mechanism is Low, two Moderate and three or more Higher. The planner also ranks the loci not yet in the stack by the gain in protection they would give. Because the weights are priors, the planner is a thinking aid for comparing stacks, not a prediction; Section 7.3 evaluates its behaviour only on simulated data.

Table 6. Ty loci, mechanisms and illustrative effect weights used by the stacking planner

Locus	Mechanism	Weight	Note
Ty-1/Ty-3	RNA-dependent RNA polymerase (DNA methylation)	0.45	Broad geminivirus resistance; reported to weaken at high temperature and under recombinant pressure.
Ty-2	NB-LRR immune receptor (TYNBS1)	0.4	Effective against TYLCV but narrower against other begomoviruses.
ty-5	Pelota loss of function (recessive)	0.35	Susceptibility-gene mechanism; recessive, so both parents must carry it.
Ty-6	DNA polymerase delta subunit 1 variant (POLD1)	0.25	Partial resistance; best when combined with another locus.
Ty-4	Minor-effect locus	0.1	Small effect on its own.

4.7.3 Insecticide rotation planner

The rotation planner generates a sequence of mode-of-action (MoA) groups from the groups the user has available. It never repeats the group used last, prefers the group least used in the last three sprays, and excludes groups for which the user has reported a control failure; if fewer than three usable groups remain it warns that resistance risk is high. The efficacy-adaptive variant (v2) accepts the latest observed field efficacy of each group and estimates the current efficacy of a group not used recently as

$$\hat{e}_g(t) = 0.8 - (0.8 - e_g) \cdot 0.95^{(age_g)} \quad (10)$$

where *e*_g is the last observed efficacy, *age*_g is the number of sprays since the group was last used, 0.8 is the assumed efficacy of a susceptible group and 0.95 is an assumed per-spray recovery factor; groups with observed efficacy below 0.5 are treated as failed. The planner gives no product or dose advice, since these depend on local registration and labels [17], [44].

4.7.4 Sequence surveillance

Sequence classification uses alignment-free k-mer containment against a reference panel that the user supplies (k = 12, circular genomes): a query is assigned to the reference with the highest fraction of shared k-mers if that fraction is at least 0.5, and is otherwise reported as unclassified. Recombination is screened by sliding windows. The v1 screen compares the k-mer containment of each window (200 nt, step 50) with every reference and flags a query when windows that are best explained by at least two different references each span at least three windows. The v2 screen uses only *discriminative* k-mers (present in some but not all references; k = 10, windows of 150 nt, step 30) so that shared k-mers, which carry no information about origin, do not dilute the signal. Both are triage tools; confirmation needs dedicated recombination analysis and phylogenetics [4], [42].

4.7.5 Calibrated incidence model (v2)

When field-trial records are available, a binomial logistic model is fitted by Newton–Raphson (iteratively reweighted least squares) with a ridge penalty of 1. For a trial with n plants and k infected, the likelihood contribution is binomial, so trials of different sizes are weighted

correctly. 7 lists the 19 features, which encode the temperature response, vector density, crop stage, each *Ty* locus, exclusion, recombinant pressure and the interactions that the literature suggests (heat and recombinants weakening *Ty-1/Ty-3*; multiple mechanisms helping).

Table 7. Features of the calibrated incidence model (v2)

Feature	Meaning
bias	intercept
tf	temperature factor f_T
tf2	f_T squared
log_wf	ln(1 + whitefly per leaf)
stage_seedling	seedling/flowering/fruiting stage indicator
stage_flowering	seedling/flowering/fruiting stage indicator
stage_fruiting	seedling/flowering/fruiting stage indicator
locus_Ty-1/Ty-3	locus present: Ty-1/Ty-3
locus_Ty-2	locus present: Ty-2
locus_ty-5	locus present: ty-5
locus_Ty-6	locus present: Ty-6
locus_Ty-4	locus present: Ty-4
exclusion	insect-proof exclusion
recomb	recombinant pressure
ty13_hot	Ty-1/Ty-3 × heat (T − 30)/5
ty13_recomb	Ty-1/Ty-3 × recombinants
multi_mech	two or more distinct resistance mechanisms

4.7.6 Seed-lot register and trial summaries

For seed lots tested with n sampled seeds and no positives, the register reports a 95% upper bound on prevalence of $3/n$ (the rule of three), because an untested or negative lot cannot be assumed free of infection when seed transmission is possible [3]. Field-trial incidence is reported by locus combination with Wilson intervals.

5. System Implementation

5.1 Architecture

TYLCV-Guard is a small web application (7) written with the Python standard library, SQLite and vanilla JavaScript, so that it can run on a modest machine without installing a web framework. A single-page interface calls a JSON API; the API validates input and delegates to domain modules; data are stored in SQLite and the calibrated model in a JSON file; the optional image-scanning path calls a hosted workflow or a local PyTorch checkpoint. 8 shows the folder structure of the repository.

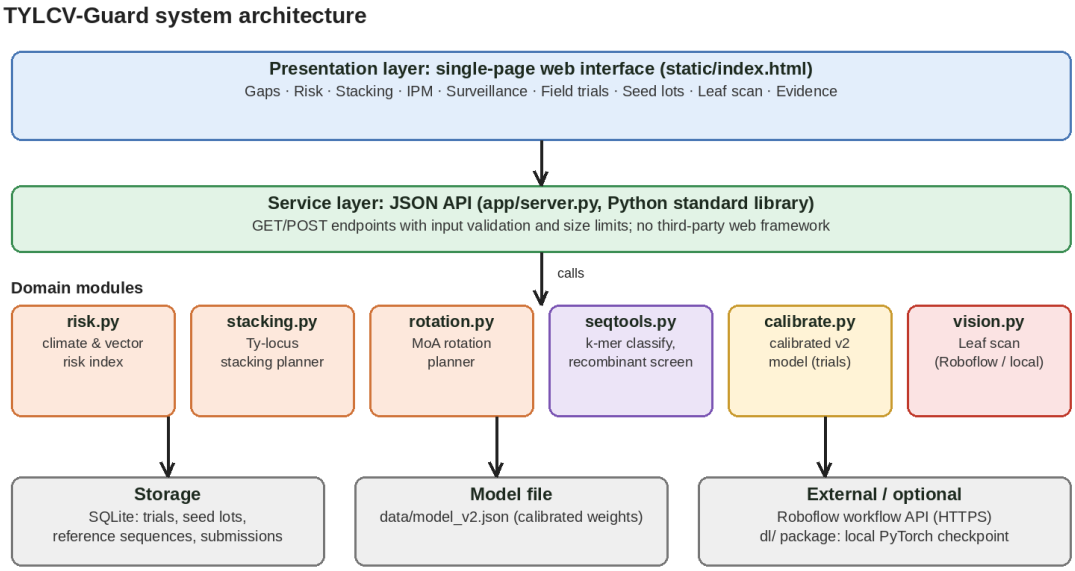


Figure 7. System architecture of the TYLCV-Guard prototype.

Project folder structure

```
tylcv-guard/
├── app/                  stdlib server + modules
│   ├── risk.py           climate & vector risk index
│   ├── stacking.py       Ty-locus stacking planner
│   ├── rotation.py       MoA rotation planner
│   ├── seqtools.py       k-mer classify, recombinant screen
│   ├── calibrate.py      calibrated v2 model (from trials)
│   ├── vision.py         Roboflow / local-model Leaf scan
│   ├── db.py, server.py SQLite storage, JSON API
├── dl/                   leaf-image deep learning
│   ├── data.py           scan, duplicate groups, splits
│   ├── model.py          backbones, Grad-CAM
│   └── train.py, infer.py   train/evaluate, predict
├── benchmark/            simulator, run.py, 15 figures
├── tools/                dataset audit, batch scan
├── static/index.html     web interface (8 tabs)
├── tests/                34 automated tests
└── README.md, Dockerfile, requirements-optional.txt
```

Figure 8. Project folder structure of the prototype.

5.2 Data model and API

8 lists the four SQLite tables, and 9 the API endpoints. All endpoints accept and return JSON, validate their input, and respond with HTTP 400 and a message for invalid requests; request bodies are limited to 8 MB, which accommodates a 5 MB image in base64.

Table 8. SQLite data model

Table	Main columns	Purpose
refseqs	name, lineage, sequence	User-supplied reference genomes for classification
submissions	name, country, collected, length, best_match, score, status, recombinant_flag	Surveillance records
trials	site, country, season, cultivar, loci, strain, mean_max_temp, whitefly_per_leaf, stage, exclusion, recombinant, plants, infected, severity	Field trials for incidence summaries and model fitting
seedlots	lot, origin, method, tested, positive, sample_size, date	Seed-lot status and prevalence bound

Table 9. API endpoints

Method and path	Function
GET /api/health, /api/gaps, /api/loci, /api/evidence	Service status; gap-to-feature map; Ty-locus table; evidence library
POST /api/risk, /api/risk/season	Risk index for one observation or a daily series
POST /api/stacking	Protection, durability and recommended next locus for a stack
POST /api/rotation	Mode-of-action rotation plan (optional efficacy map)
GET/POST /api/refseqs; POST /api/classify; GET /api/surveillance/summary	Reference panel, sequence classification and recombinant screening, country summary
GET/POST /api/trials; GET /api/trials/summary, /api/trials.csv	Field-trial log, incidence by locus combination, CSV export
GET /api/model; POST /api/model/train, /api/model/predict	Calibrated model status, training from trials, prediction
GET/POST /api/seedlots	Seed-lot register
GET /api/vision/status; POST /api/diagnose	Leaf scan configuration and image diagnosis

5.3 Web interface

The interface has eight tabs. The *Gaps* tab (9) lists the research gaps identified in the literature review and the feature that addresses each, which keeps the prototype tied to the problem statement. The *Risk* tab (10) computes the risk index for one observation and for a daily series. The *Stacking* tab (11) evaluates the protection and durability of a chosen set of *Ty* loci and recommends the best additions. The *IPM* tab (12) produces a rotation plan. The *Surveillance* tab (13) manages the reference panel, classifies sequences and screens for recombinants. The *Field trials* tab (14) logs trials, summarises incidence with Wilson intervals, exports CSV and trains and queries the calibrated model. The *Seed lots* tab (15) records seed-lot tests, and the *Leaf scan* tab (16) sends a photograph to the image model. Figures in this section were produced with demonstration data generated by the simulator and do not represent real trials, sequences or seed lots.

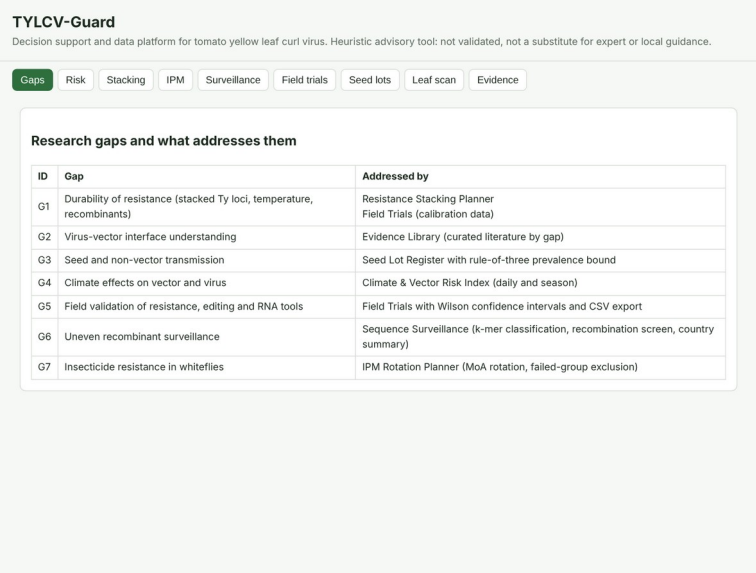


Figure 9. Gaps tab: research gaps and the features that address them.

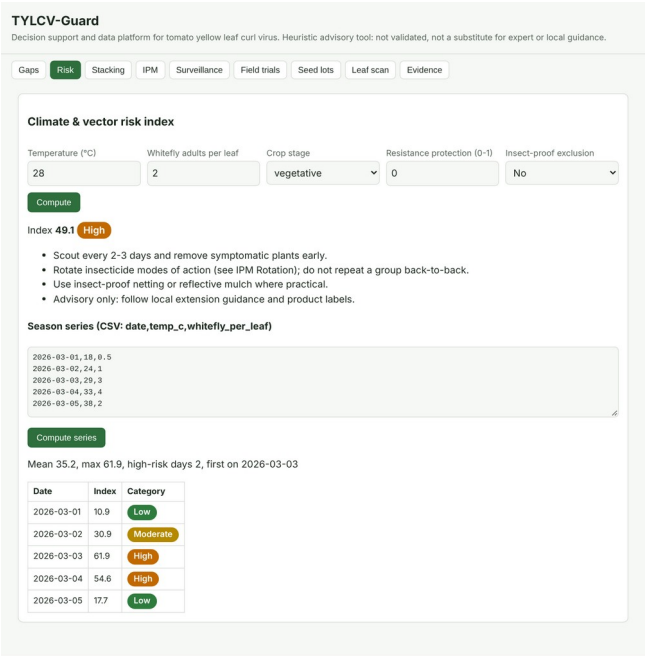


Figure 10. Risk tab: risk index for one observation and for a season series (demonstration data).

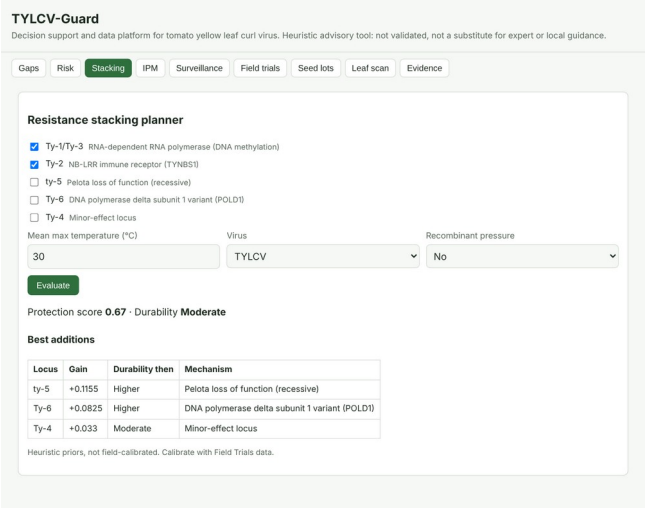


Figure 11. Stacking tab: protection score, durability tier, warnings and recommended additions for a chosen stack.

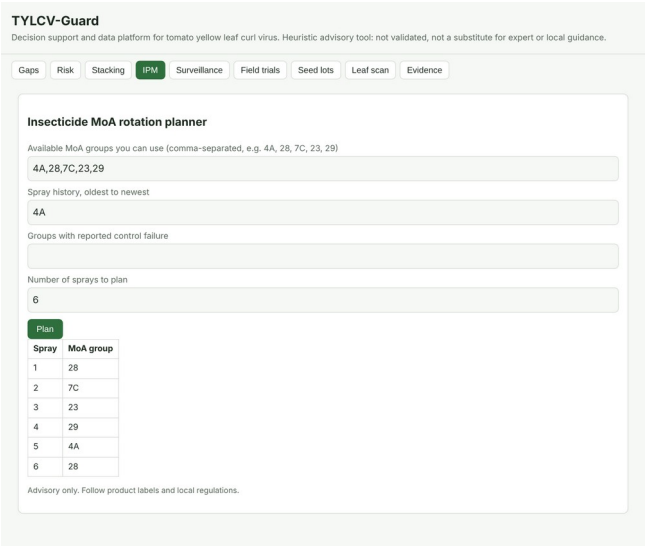


Figure 12. IPM tab: insecticide mode-of-action rotation plan.

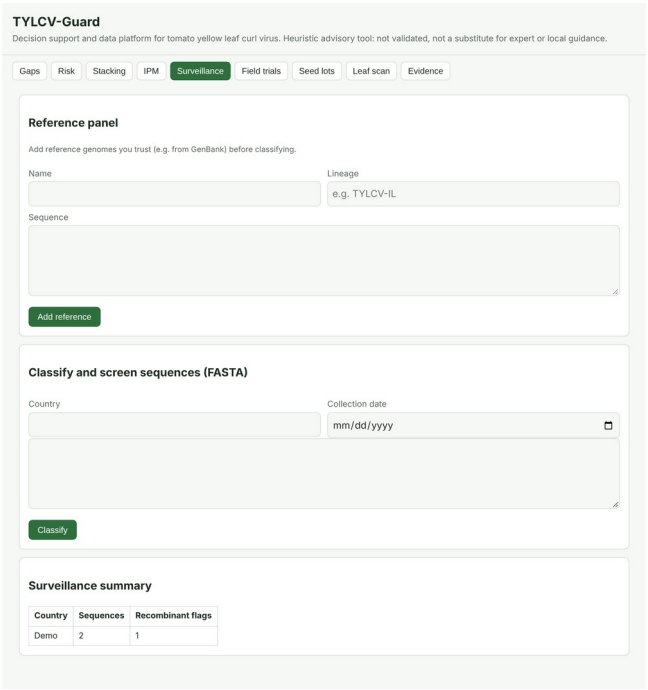


Figure 13. Surveillance tab: reference panel, classification and recombinant screening (synthetic sequences).

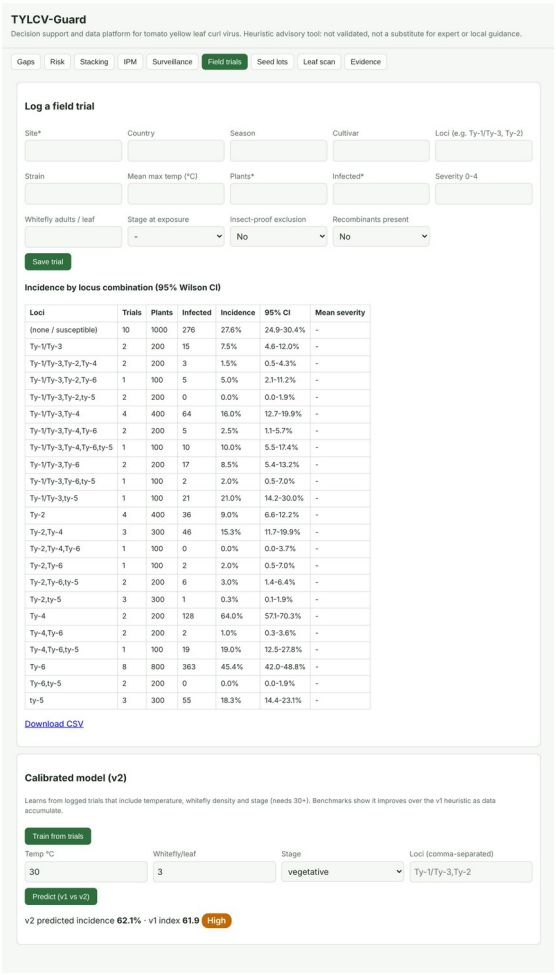


Figure 14. Field trials tab: incidence by locus combination with 95% Wilson intervals, and the calibrated model panel (synthetic demonstration trials).

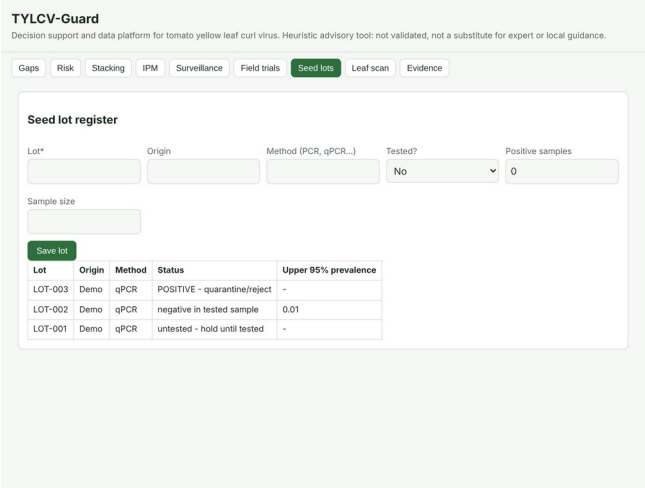


Figure 15. Seed lots tab: lot status and the rule-of-three prevalence bound (demonstration data).

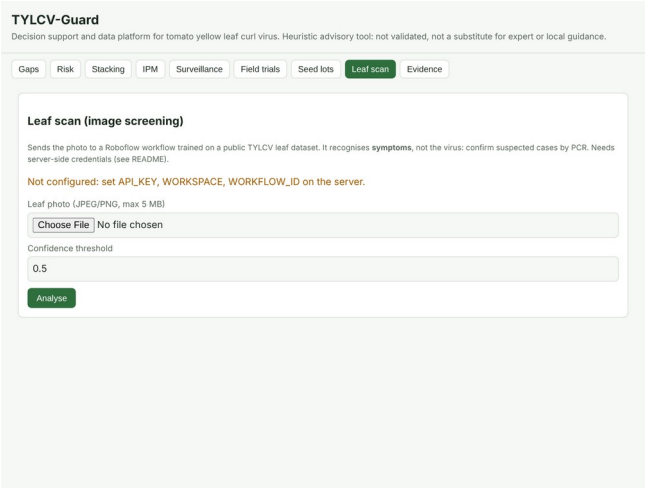


Figure 16. Leaf scan tab: shown here without credentials, so the service reports that it is not configured.

5.4 Leaf scan integration

The Leaf scan path (17) accepts a base64 JPEG or PNG, validates the magic bytes and the 5 MB limit, writes the bytes to a temporary file, calls a Roboflow workflow through the inference SDK or, when a local checkpoint is configured, runs the PyTorch model of the `dl` package, and returns a verdict, a confidence and advice. Credentials are read only from environment variables and are sent in a header; the temporary file is deleted after the call; SDK errors are replaced with a generic message so that keys cannot leak. The hosted workflow is based on a public Roboflow Universe dataset [49]; because the service was not reachable from our development environment, this path was tested only with a mocked client and has not been exercised against the real service. The tool states that a positive result is a screening flag that must be confirmed by PCR or qPCR.

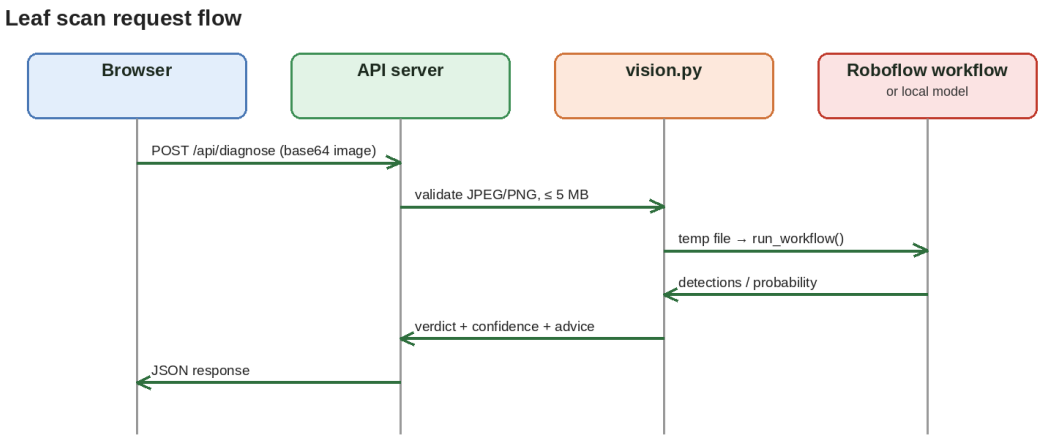


Figure 17. Leaf scan request flow.

5.5 Deep-learning package

The `dl` package contains data scanning and group splitting (`data.py`), the model factory and Grad-CAM (`model.py`), the training and evaluation command (`train.py`) and inference (`infer.py`). Training writes the checkpoint, a metrics file with test and external-set results, an evaluation figure and an automatically generated model card. The command refuses to train when any split lacks one of the two classes, which prevents a diagnostic model from being trained on positives alone.

5.6 Testing

Automated tests cover input validation, monotonic behaviour of the risk index, stacking, rotation constraints, sequence classification and recombination screening, the database layer, the API, the calibrated model, the vision integration with a mocked client and the deep-learning pipeline (10, 18). The deep-learning tests use synthetic images and show that the pipeline runs end to end and that near-duplicates do not cross splits; they do not say anything about accuracy.

Table 10. Inventory of automated tests

Test class	Tests	Covers
API	3	HTTP endpoints and error handling
Calibrated	4	calibrated model, efficacy-adaptive rotation, v2 screen
DB	4	Wilson interval, trials, seed lots, surveillance flow
ModelAPI	1	model training and prediction through the API
Risk	4	risk index: shape, monotonicity, validation, season series
Rotation	3	rotation planner: no consecutive repeats, exclusion of failed groups
Seq	4	FASTA parsing, classification, recombination screen
Stacking	4	stacking planner: protection, heat and recombinant penalties, durability
Vision	4	Leaf scan with a mocked client, validation, sanitised errors
DL	3	deep-learning pipeline on synthetic images
Total	**34**	

Automated test run

```
$ python -m unittest discover -s tests -t . -v
test_bad_json ... ok
test_endpoints ... ok
test_pages_and_data ... ok
test_efficacy_rotation ... ok
test_fit_learns_direction ... ok
test_too_few_rows ... ok
test_v2_screen ... ok
test_seedlots ... ok
test_surveillance_flow ... ok
test_trials ... ok
...
test_sdk_error_is_sanitised ... ok
test_validation ... ok
test_workflow_called_with_config_and_cleanup ... ok
test_labels_and_split_without_leakage ... ok
Ran 34 tests in 22.610s
OK
```

Figure 18. Console output of the automated tests.

5.7 Deployment and reproducibility

The application runs with `python -m app.server`, needs no packages beyond the Python standard library, and ships with a Dockerfile. Optional packages (PyTorch, the Roboflow inference SDK, scikit-learn and Matplotlib) are needed only for the deep-learning package, the hosted Leaf scan and the experiments. All experiments reported in this paper are scripts in the repository with fixed random seeds (Appendix C).

6. Experimental Design

6.1 Types of evidence and how they are used

The study combines three kinds of evidence, and we keep them apart throughout because they support different claims. *Real* evidence is an analysis of real photographs. *Synthetic-image* evidence comes from experiments that train real models on artificially generated images, to demonstrate a mechanism under controlled conditions. *Simulated* evidence comes from a generative model of fields and infections, used to test the decision-support components when no field data are available. 11 summarises the experiments.

Table 11. Experiments, evidence type and purpose

Experiment	Data	Evidence type	Purpose	RQ
Audit of public TYLCV folder	100 real photographs	Real (descriptive)	What the data do and do not allow	RQ1
Leakage experiment	Synthetic leaf images with near-duplicates	Synthetic images, real training	Inflation of accuracy by random splits	RQ2
Risk-model benchmark	Simulated fields (3,600 + 2,400 + 1,200)	Simulated	Compare previous approaches, v1 and v2	RQ3
Sensitivity analyses	Simulated fields	Simulated	Bootstrap CIs, noise, training size, thresholds	RQ4
Module benchmarks	Simulated sequences, simulated resistance dynamics, analytic bound	Simulated / analytic	Stacking, recombinant screen, rotation, seed bound	RQ3
Software verification	Unit and integration tests	Software	Correct, reproducible implementation	RQ5

6.2 The public TYLCV image folder

The real data are a public Kaggle image folder of TYLCV leaves and plants [50], which was supplied to us as an archive. It was extracted into an isolated directory and treated as untrusted data: only image files were read, nothing was executed from it, and the archive paths were checked before extraction. We did not verify the dataset's licence, so the photographs shown in this paper are used only to illustrate the audit and should not be redistributed without checking it. The folder contains no labels beyond the fact that it is a TYLCV collection, so every image is treated as positive. A second public collection, a Roboflow Universe dataset [49], was not accessible from our environment and was not audited.

6.3 Synthetic leakage experiment

To measure how much a random split can inflate accuracy, we generated synthetic leaf images (64 × 64 pixels) in two classes. Each image shows a randomly coloured background with random clutter and a leaf-like ellipse whose colour has a class-dependent offset (16 units on the red channel and 13 on the green channel for the positive class) that is small compared with per-image colour variation (standard deviation 28 plus 16). Each of 70 base images per class was expanded into four near-duplicate variants by a small brightness change, a one-pixel shift and rescaling through 56 or 72 pixels followed by resizing back, giving 560 images per seed. The signal is deliberately weak: an honest model should find the task hard, and a model that memorises individual images will look good only if copies of the test images are in the training set.

Each dataset was split in two ways: a random image-level split (70/15/15) and the duplicate-aware group split of Section 4.2, using a threshold of 20 bits because variants of these synthetic images differ more than the real duplicates in Section 7.1. We trained (a) a MobileNetV3-Small network from random initialisation for 50 epochs without augmentation, using the last-epoch weights so that it can fit the training set, for 3 seeds; and (b) three classical learners on

16 × 16 RGB pixels (1-nearest-neighbour, 5-nearest-neighbour and logistic regression) for 20 seeds. For each split we also computed the leak fraction of Equation 3, using the known base-image identifiers.

6.4 Simulated field benchmark

The simulator generates field-season records. A region (cool, warm or hot, with mean temperatures 22, 27 and 32 °C) determines the temperature, which is drawn with a standard deviation of 3 °C; whitefly density is log-normally distributed; crop stage, the set of *Ty* loci, exclusion and recombinant pressure are drawn at random; and a field random effect is added. The infection probability is a logistic function of a Gaussian temperature response around 27 °C, a Hill function of whitefly density, additive stage effects, additive locus effects that are weakened by heat and recombinants for *Ty-1/Ty-3*, a bonus for multiple mechanisms, a strong effect of exclusion and the random effect. The observed incidence is the proportion infected among 100 sampled plants. An *outbreak* is an incidence of at least 30%.

We deliberately used functional forms that differ from those of the heuristic model (Gaussian rather than piecewise-linear temperature response, Hill function rather than logarithm, additive logit rather than multiplicative effects), so that the heuristic is a mis-specified approximation rather than the data-generating process. Nevertheless, both the simulator and the models encode the same qualitative statements from the literature (heat and recombinants weaken *Ty-1/Ty-3*, young plants are more susceptible), and the author of the simulator also designed the models; the benchmark therefore shows how components behave if those statements are right, not whether they are right. In the main benchmark, 3,600 fields were split into 2,400 for training and 1,200 for testing, and a second test set of 1,200 hot-region fields was used with training restricted to cool and warm regions to probe a mild distribution shift.

6.5 Compared methods

Previous single-factor approaches are a temperature-only score (the temperature factor), a whitefly-threshold score (adult density) and a temperature × whitefly score. The first heuristic model (v1) is the risk index of Section 4.7.1 with the protection score of the stacking planner. The calibrated model (v2) is the logistic model of Section 4.7.5 trained on the training fields. To make the comparison fair, each single-factor score and v1 are recalibrated on the same training data by a one-variable binomial logistic regression, so that all methods output a predicted incidence.

6.6 Metrics and statistical procedures

Outbreak classification is evaluated by ROC-AUC and average precision; incidence prediction by mean absolute error (MAE), root-mean-square error and the expected calibration error over deciles of the predictions. Uncertainty of AUC is estimated by bootstrap resampling of the test set (1,000 resamples). Sensitivity to observation noise varies the number of sampled plants per

field (10 to 200; 3 seeds), and sensitivity to training size varies the number of training fields (30 to 1,000; 5 seeds). Decision thresholds are analysed with the expected-cost rule of Equation 6 for several cost ratios.

6.7 Computing environment

Table 12. Computing environment and software versions

Item	Value
Hardware	4 CPU cores, 15 GB RAM, no GPU
Operating system	Linux 6.x
Language and libraries	Python 3.13; PyTorch 2.15; torchvision 0.29; NumPy 2.5; scikit-learn 1.9; Matplotlib 3.11
Application dependencies	Python standard library and SQLite only
Randomness	Fixed seeds for every experiment (see Appendix C)

7. Results

7.1 Audit of the public TYLCV image folder (RQ1)

7.1.1 Composition

The folder contains 100 JPEG files, none corrupt. Grouping by file name and dimensions gives three sources (19, 13): 91 images of 256 × 256 pixels with PlantVillage-style names (lab-style images, a single leaf on a plain background), 7 phone photographs of 3000 × 4000 pixels, and 2 photographs of 960 × 1280 pixels. There are no exact duplicates. All 100 images are TYLCV-positive: the folder has no healthy leaves and no other disease, so neither accuracy, nor specificity, nor calibration can be estimated from it.

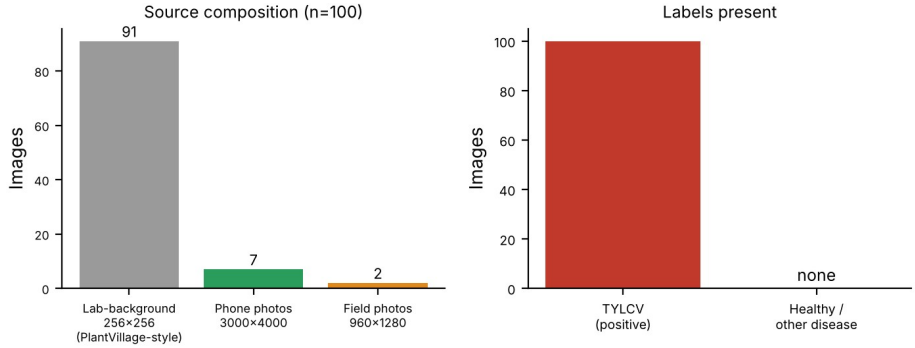


Figure 19. Audit of the public TYLCV folder: source composition and label coverage.

Table 13. Audit summary and mean image statistics by source (means over images)

Property	Lab-style	Phone photo	Field photo (960×1280)
Images	91	7	2
Dimensions (pixels)	256 × 256	3000 × 4000	960 × 1280
Leaf coverage (fraction of pixels)	0.382	0.040	0.032
Mean saturation	0.289	0.133	0.137
Sharpness (Laplacian variance)	535	5129	4064
Background brightness (border ring)	0.508	0.542	0.566
Leaf yellowness index	0.225	0.114	0.143
File size (KB)	12	3480	223
Label coverage	100% TYLCV	100% TYLCV	100% TYLCV

7.1.2 What the photographs look like

20 shows twelve of the lab-style images. Each shows one leaf or leaflet on a uniform grey or brown backdrop, with strong curling and mottled yellowing visible on many leaves. 21 shows the nine other photographs. They are quite different: the leaf occupies a small part of the frame, the background is soil, weeds and other leaves, the lighting is uneven and several leaves are partly dry or senescent, which a model would have to distinguish from TYLCV symptoms.



Figure 20. Twelve lab-style images from the audited folder (256 × 256 pixels). Photographs are shown for illustration only; the dataset licence was not verified.



Figure 21. The seven phone photographs (3000 × 4000) and two field photographs (960 × 1280) of the audited folder, shown at reduced size.

7.1.3 Near-duplicates

Average-hash distances between all 4,950 image pairs (22) have a clear gap: only 2 pairs are closer than 12 bits, and the next-closest pair of images is 21 bits apart. The two close pairs are the same photographs stored at two resolutions (23): each 960 × 1280 photograph is a downscaled copy of a 3000 × 4000 phone photograph, at hash distances of 7 and 1 bits. A random split could therefore place one copy in training and the other in testing, and two of the nine non-lab photographs would not be independent test cases. This is the situation that the group split of Section 4.2 handles.

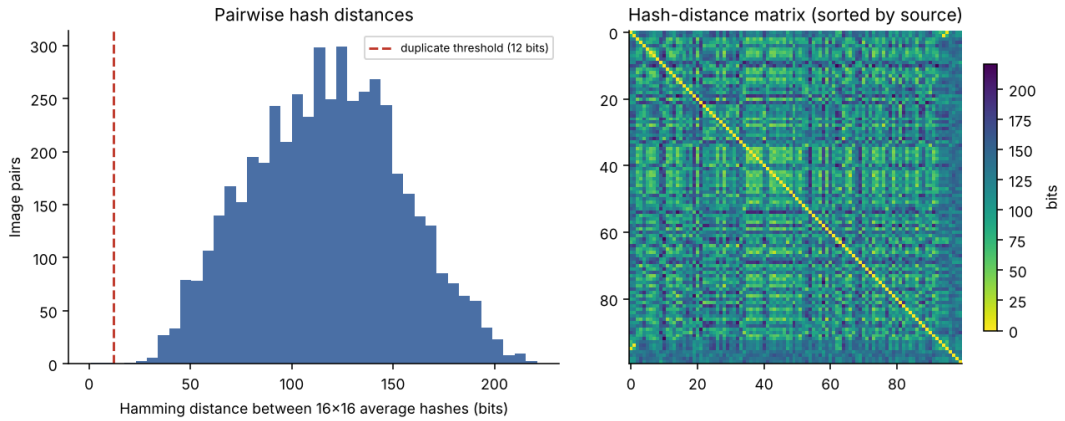


Figure 22. Pairwise average-hash distances: histogram with the 12-bit duplicate threshold, and the distance matrix sorted by source.



Figure 23. The two near-duplicate pairs: the same photograph at two resolutions.

7.1.4 Low-level image statistics and shortcut risk

The two kinds of photographs differ strongly in simple statistics (24, 13). The leaf covers about 38.2% of a lab-style image but only 4.0% and 3.2% of the phone and field photographs; the lab-style images are far more saturated; and the large photographs are much sharper at the analysis scale and thousands of times larger as files. The smallest leaf coverage among lab-style images (0.223) exceeds the largest among the other photographs (0.117), so a single threshold on leaf coverage separates the two groups without error. A logistic regression on three such statistics (leaf coverage, saturation and sharpness) classified the source with a leave-one-out accuracy of 100.0% and recovered 100.0% of the non-lab photographs. A principal-component projection of ten colour and sharpness features (25) shows the same separation.

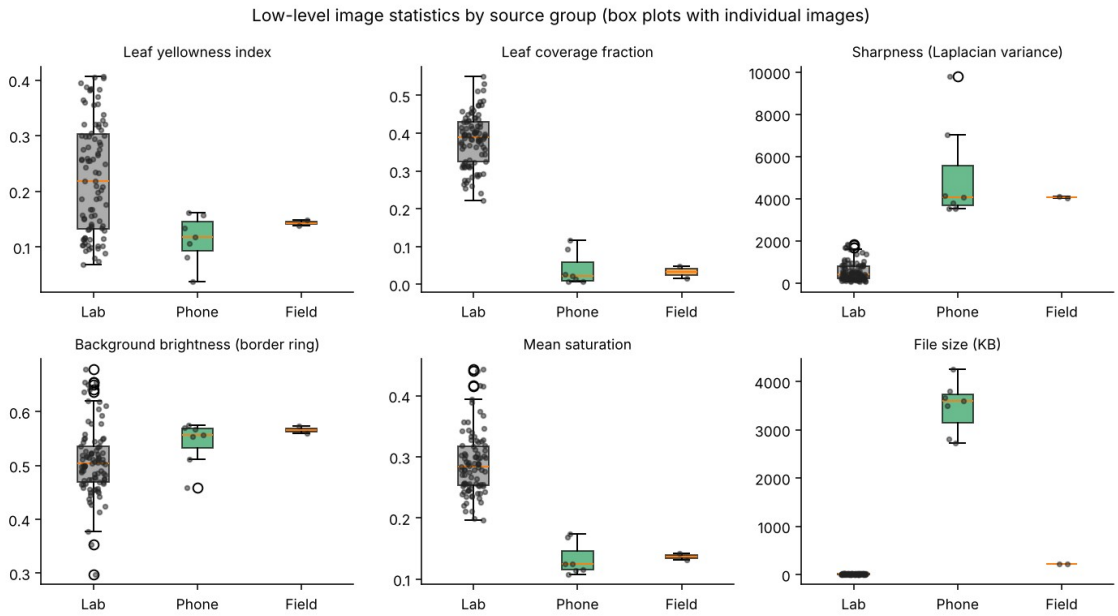


Figure 24. Low-level image statistics by source group, as box plots with the individual images overlaid.

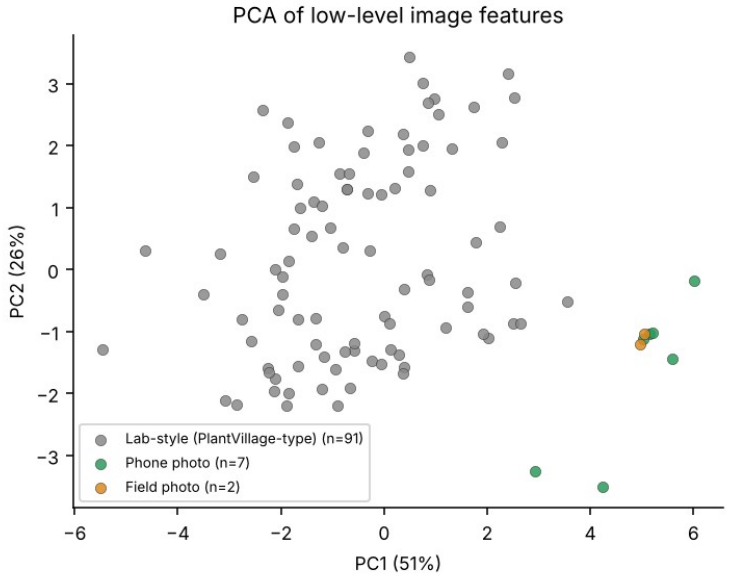


Figure 25. Principal-component projection of ten low-level features, coloured by source.

7.1.5 Implications

These findings answer RQ1. The folder cannot support an estimate of diagnostic accuracy because it has no negatives. It is also a poor basis for training even a screening model by combining it with negatives from another source: if healthy leaves come from a lab-style collection and TYLCV photographs from the field, a classifier can separate them by leaf coverage alone, exactly the kind of shortcut discussed in the literature [12], [13]. Conversely, if the negatives are also lab-style, a model trained on 91 lab-style positives would be tested on lab-style data and tell us little about the nine field photographs. The nine non-lab photographs, of which only seven are independent, are the only real evidence of field conditions, and they are too few to evaluate a model. Section 8 discusses what data are needed.

7.2 Leakage experiment (RQ2)

14 and 26 report test accuracy (mean ± standard deviation over seeds; chance is 0.5) for the two splits. With a random split, 96.7% of test images have a near-duplicate in the training set; with the duplicate-aware split the leak fraction falls to 19.2%, not to zero, because some variants differ by more than the hash threshold allows (we return to this in Section 9). Under the random split, 1-nearest-neighbour (0.98), logistic regression (0.94) and the CNN (0.90 on average) score highly, and 5-nearest-neighbour 0.78, whereas under the duplicate-aware split all fall to between 0.54 and 0.62 (the CNN averages 0.60 but is variable, 0.46 to 0.72 over its three seeds). The remaining margin above chance under the group split is consistent with the residual leak fraction and a small genuine signal; the experiment cannot separate the two. For example, 1-nearest-neighbour on pixels drops from 0.983 ± 0.019 to 0.600 ± 0.114.

Table 14. Test accuracy under random and duplicate-aware splits (synthetic images; mean ± standard deviation)

Learner	Random split	Duplicate-aware split	Difference
1-NN (pixels)	0.983 ± 0.019	0.600 ± 0.114	+0.383
5-NN (pixels)	0.784 ± 0.049	0.544 ± 0.114	+0.240
Logistic (pixels)	0.942 ± 0.024	0.615 ± 0.127	+0.326
MobileNetV3-Small (3 seeds)	0.901 ± 0.044	0.599 ± 0.108	+0.302

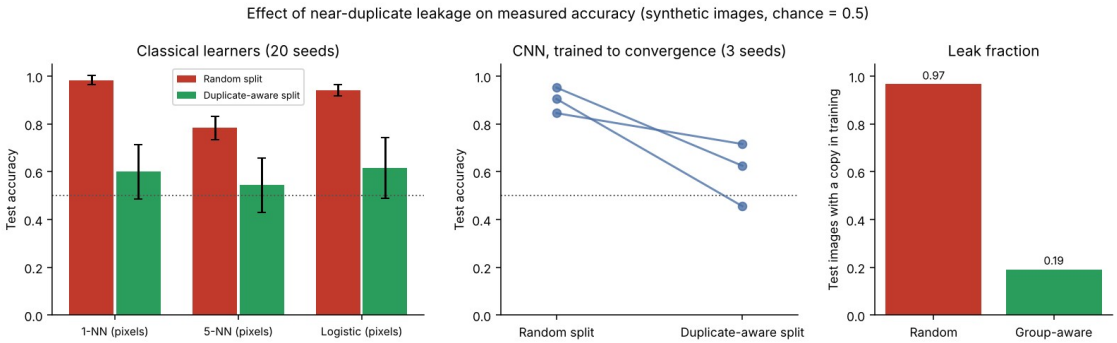


Figure 26. Effect of near-duplicate leakage on measured accuracy: classical learners, the CNN and the leak fraction.

This experiment answers RQ2 for the setting studied. The signal in these images is weak by construction, so the models cannot learn it from 390 training images, and what the random split rewards is memorisation of the copies. That is the mechanism by which duplicates inflate reported accuracy. The size of the inflation here (tens of percentage points) is a property of this synthetic setting and should not be transferred to real data; what transfers is the direction of the effect and the need to measure and report the leak fraction.

7.3 Simulated benchmark: decision-support components

7.3.1 Outbreak and incidence prediction (RQ3)

15 reports performance on 1,200 held-out simulated fields. The single-factor baselines reached an AUC of 0.635 to 0.764, the heuristic model 0.930 and the calibrated model 0.980; the mean absolute error of incidence fell from 0.117 for the best single-factor model to 0.074 and 0.038. The ROC and precision–recall curves (27) show the same ordering across thresholds. On the hot-region test set unseen in training, the ordering was unchanged (28). Calibration was best for the calibrated model (29), and the distribution of absolute errors shows the same pattern (30).

Table 15. Outbreak prediction and incidence error on simulated fields (outbreak = incidence ≥ 30%)

Model	AUC	Avg. precision	MAE	ECE	AUC (hot, unseen)	MAE (hot, unseen)
Temp-only	0.635	0.222	0.126	0.069	0.666	0.106
Whitefly-threshold	0.736	0.314	0.125	0.041	0.732	0.114
Env-only (T×WF)	0.764	0.348	0.117	0.017	0.775	0.102
v1 (heuristic)	0.930	0.786	0.074	0.022	0.938	0.064
v2 (calibrated)	**0.980**	**0.897**	**0.038**	**0.003**	**0.982**	**0.036**

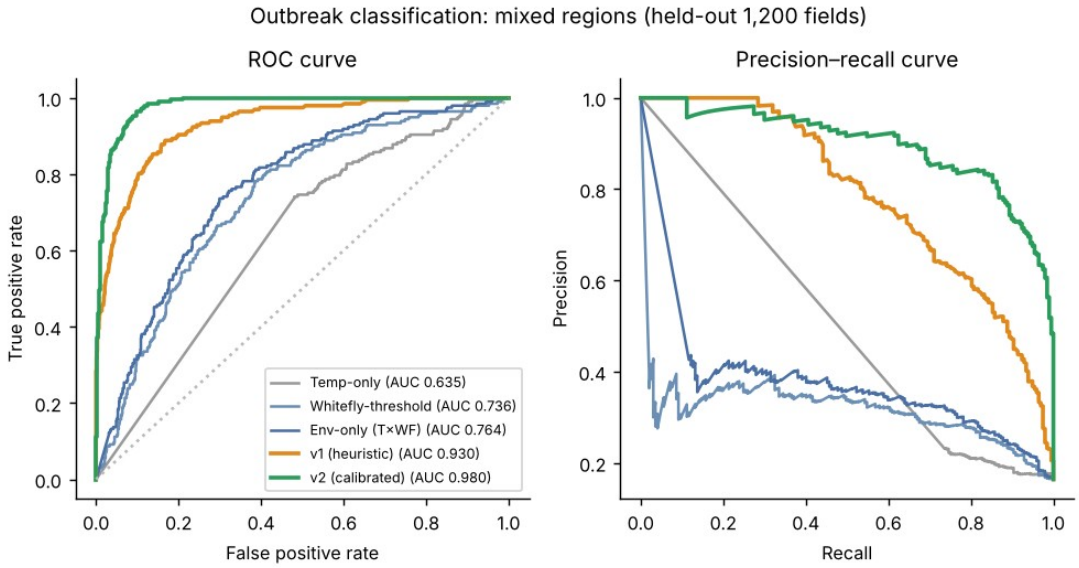


Figure 27. ROC and precision–recall curves of the risk models on held-out simulated fields.

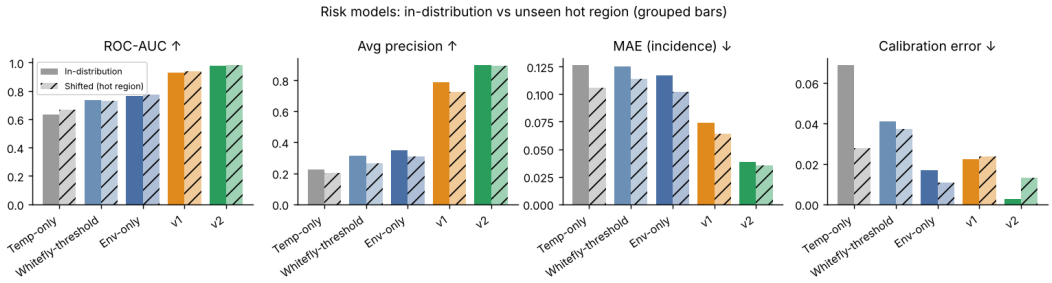


Figure 28. Metrics for in-distribution fields and for the unseen hot region (grouped bars).

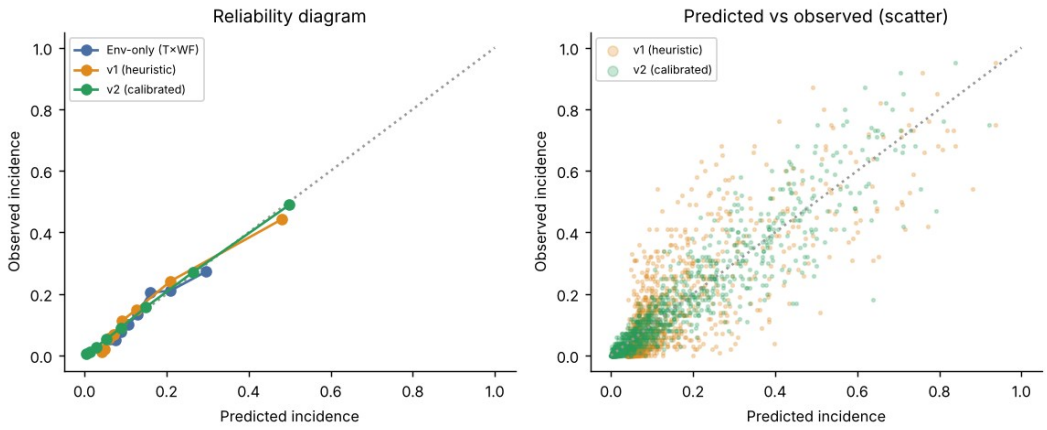


Figure 29. Reliability diagram and predicted-versus-observed scatter.

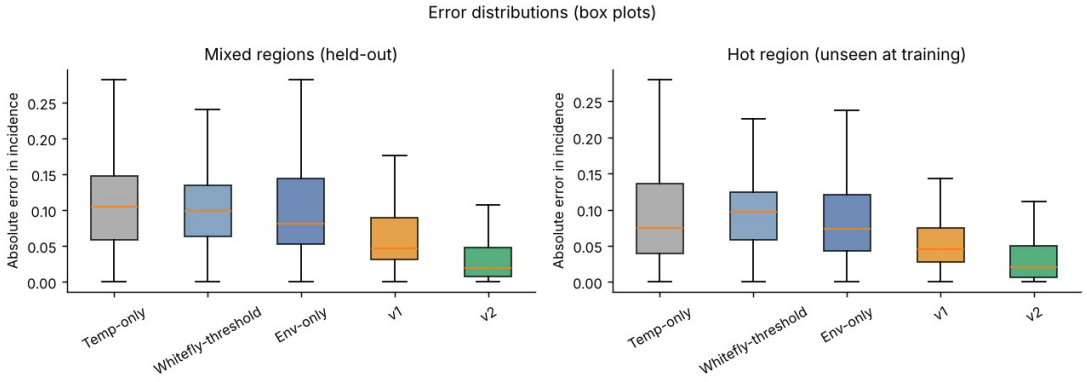


Figure 30. Distribution of absolute incidence errors (box plots).

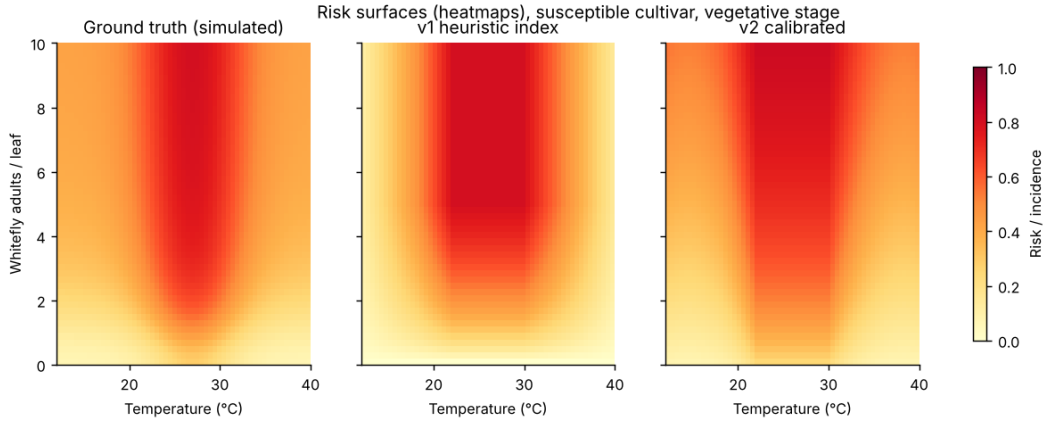


Figure 31. Risk surfaces (heat-maps): ground truth, v1 heuristic and v2 calibrated model for a susceptible cultivar at the vegetative stage.

The risk surfaces (31) show where the heuristic departs from the simulated truth: the piecewise-linear temperature response and the logarithmic vector response give a smoother and somewhat different surface than the Gaussian-Hill form of the simulator, and the calibrated model recovers the simulated surface closely. The comparison is tilted in favour of v1 and v2 over the single-factor baselines, because those ignore resistance and stage by construction, and in favour of v2 because the simulator and the model share qualitative assumptions (Section 6.4).

7.3.2 Uncertainty of the comparison

The bootstrap intervals (32) show that the differences between model groups are large relative to sampling uncertainty: the AUC of the calibrated model was 0.980 (95% interval 0.973–0.985) and that of v1 0.930 (0.910–0.947). The mean difference between v2 and v1 was 0.050 with a 95% interval of 0.035 to 0.067, which excludes zero, whereas the single-factor baselines were all far below (0.763 for the best of them). These intervals reflect only the finite test set of 1,200 fields, not uncertainty about the simulator.

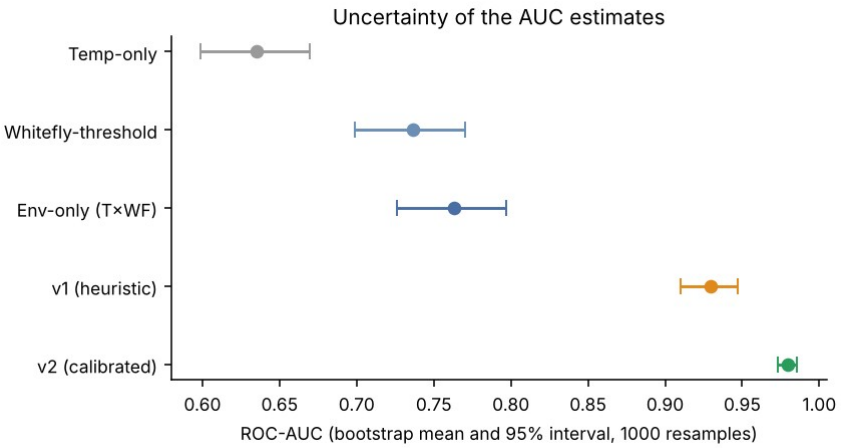


Figure 32. Bootstrap means and 95% intervals of the ROC-AUC (1,000 resamples).

7.3.3 Training size and learning curve (RQ4)

The calibrated model needs data. In the single-seed learning curve (33) it matched the heuristic with about 30 simulated trials (AUC 0.924) and exceeded it from about 60 (AUC 0.969). With five seeds (34), the mean AUC rose from 0.958 at 30 training fields to 0.978 at 1,000, with a standard deviation of 0.011 at the smallest size and 0.003 at the largest, and the calibration error fell from 0.030 to 0.004. Gains flatten after a few hundred trials. This supports the design decision to log field trials, with the caveat that real trials have noise, confounding and missing values that the simulator does not capture.

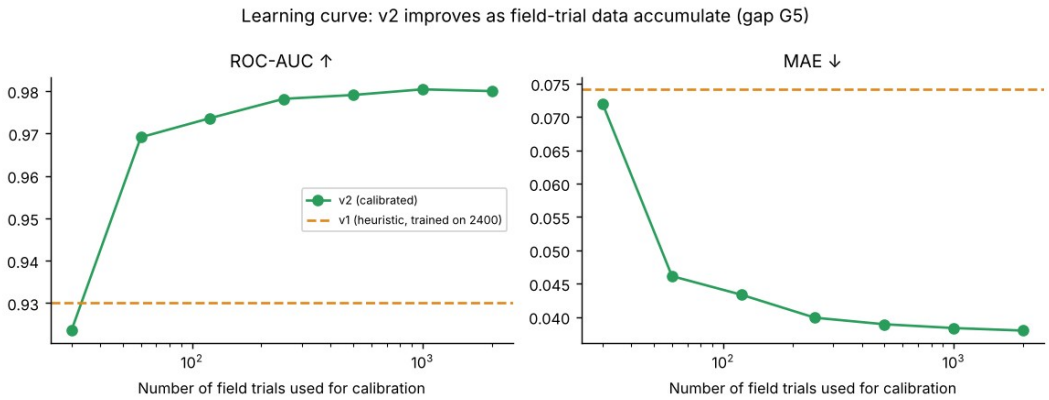


Figure 33. Learning curve of the calibrated model against the heuristic (single seed).

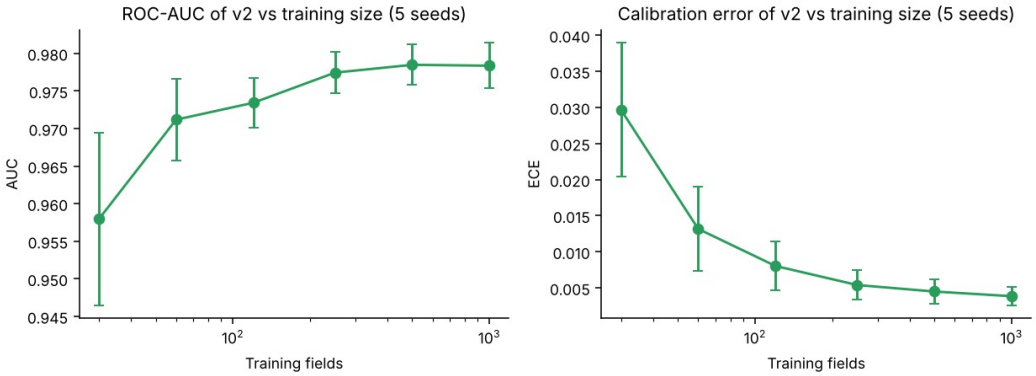


Figure 34. AUC and calibration error of the calibrated model against training size (5 seeds; mean ± standard deviation).

7.3.4 Observation noise (RQ4)

Scoring fewer plants per field makes observed incidence noisier. 35 shows that the ordering of the models was unchanged across 10 to 200 sampled plants: at 10 plants the AUC of v2 was 0.945 and that of v1 0.879, and at 200 plants 0.984 and 0.940. The advantage of the calibrated model was therefore not an artefact of the sampling design.

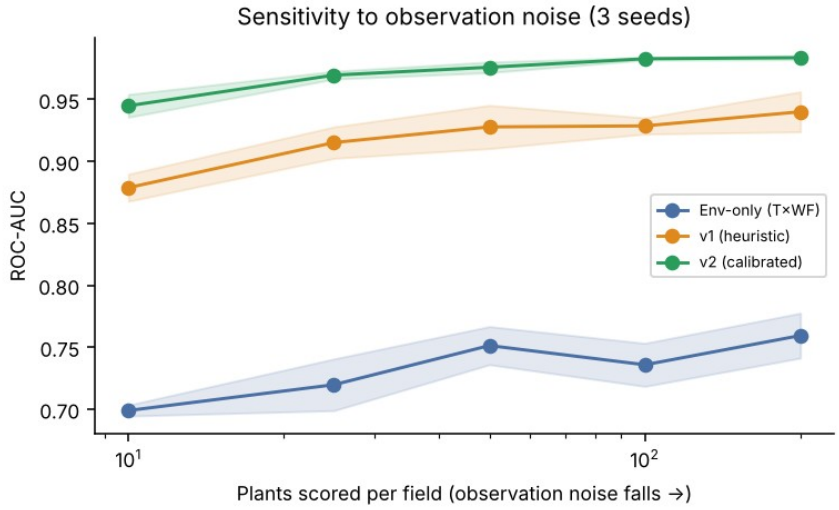


Figure 35. Sensitivity of ROC-AUC to the number of plants scored per field (3 seeds).

7.3.5 What the calibrated model learned

36 shows the learned coefficients. Exclusion has the largest protective coefficient (−1.89), followed by the *Ty-1/Ty-3* (−1.37) and *Ty-2* (−1.29) loci and the fruiting stage (−1.23); whitefly density increases risk (+1.09 per unit of ln(1 + density)) and so does the seedling stage relative to the vegetative baseline (+0.51). The two temperature terms have opposite signs (−1.91 and +3.04) because the temperature factor and its square are strongly correlated; together they give a convex increase in risk as the temperature factor rises. The interaction terms that encode the weakening of *Ty-1/Ty-3* by heat (+0.62) and by recombinants (+0.50) recover both the sign and, approximately, the size of the effects built into the simulator (0.6 and 0.5), and the multiple-mechanism term (−0.36) the simulated bonus of −0.4. Because the

simulator was designed with these effects, this is a check that the fitting procedure recovers known structure, not evidence about real resistance genes.

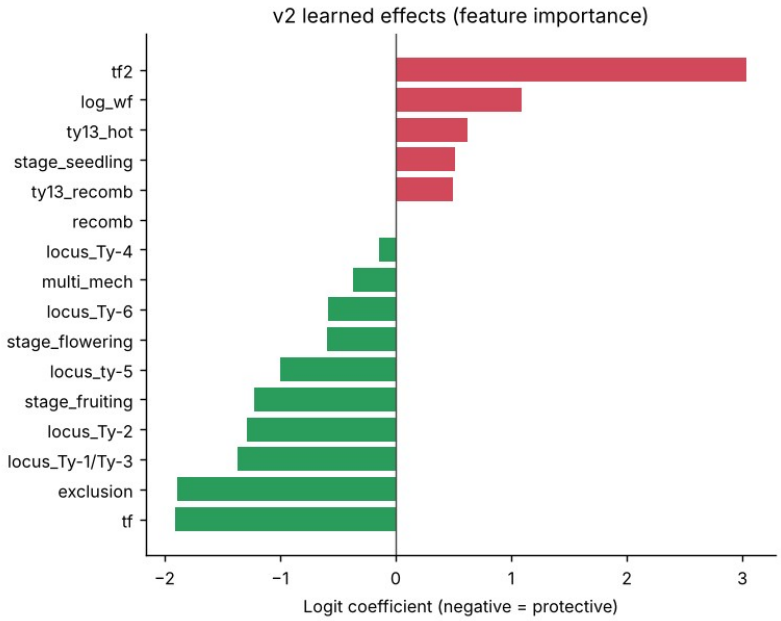


Figure 36. Learned logit coefficients of the calibrated model (negative values are protective).

7.3.6 Decision thresholds

The best threshold depends on the cost of errors (37). Minimising expected cost gave thresholds of 0.29, 0.23 and 0.18 for false-negative to false-positive cost ratios of 1, 3 and 10: the more costly a missed infection, the lower the threshold. Sensitivity and specificity move in opposite directions along the threshold, so a single accuracy figure hides a choice that should be made explicitly. For TYLCV, where a missed infection allows spread but a false alarm costs a laboratory test, a ratio greater than 1 is plausible, but the right ratio is an agronomic and economic judgement.

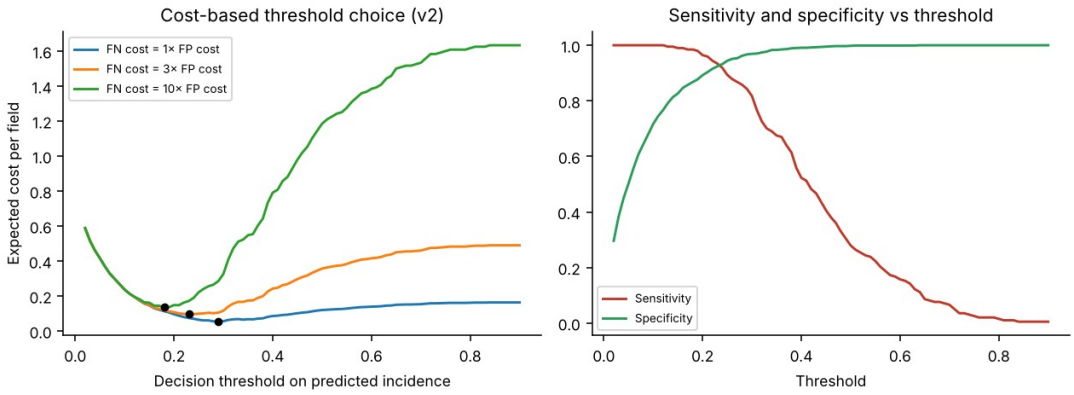


Figure 37. Expected cost per field against the decision threshold for three cost ratios, and sensitivity and specificity against the threshold.

7.3.7 Resistance stacking planner

The stacking planner was compared with a naive count of loci on the 32 possible combinations of five loci, using the simulator's true protection as reference (16, 38, 39). Both the planner and the calibrated model ranked combinations almost perfectly against the simulated truth, whereas the locus count ranked them less well and picked the best next locus correctly in only 31–38% of cases under heat or recombinants. As noted, the simulator shares the heat and recombinant assumptions of the planner, so the near-perfect agreement is expected and does not validate the weights.

Table 16. Stacking planner against the simulated truth (32 locus combinations)

Environment	Rank correlation: count / v1 / v2	Best-next-locus accuracy: count / v1 / v2
Cool, no recombinants	0.89 / 1.00 / 1.00	1.00 / 1.00 / 1.00
Hot, no recombinants	0.89 / 1.00 / 1.00	0.38 / 1.00 / 1.00
Hot + recombinants	0.85 / 0.99 / 1.00	0.31 / 1.00 / 1.00

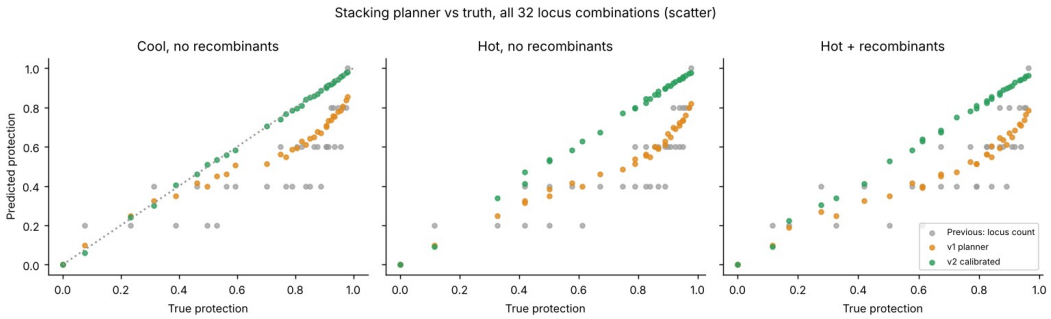


Figure 38. Predicted against true protection for all 32 combinations in three environments.

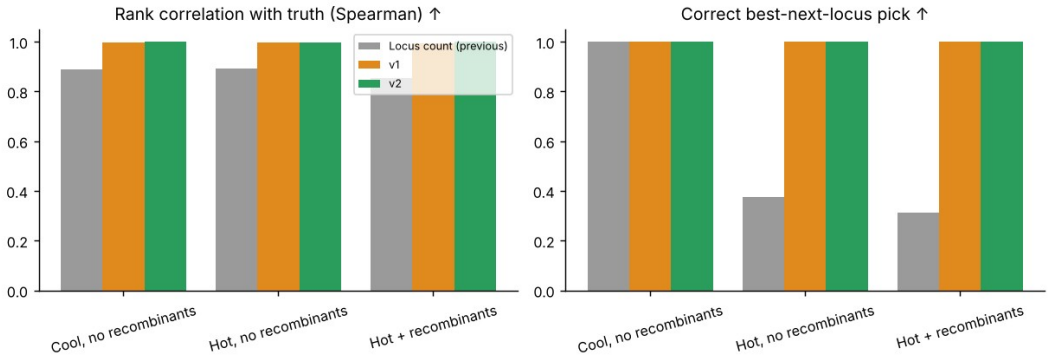


Figure 39. Rank correlation and best-next-locus accuracy of the locus count, the v1 planner and the v2 calibrated model.

7.3.8 Recombinant screen

In a deliberately hard regime (parent divergence 0.5–16%, recombinant fragments of 100–600 nucleotides, 1.5% query noise; 8 replicates per cell) the mean sensitivity was 0.00 for best-match classification (which cannot flag recombinants), 0.67 for the v1 window screen and 0.80 for the v2 screen, at roughly one percent false positives (40, 41). Both screens failed at 0.5%

divergence and for fragments of 100 nucleotides or less, where the information in a short window is simply too low. These sequences were random with independent substitutions, which is easier than real genomes with hotspots, indels and more than two parents.

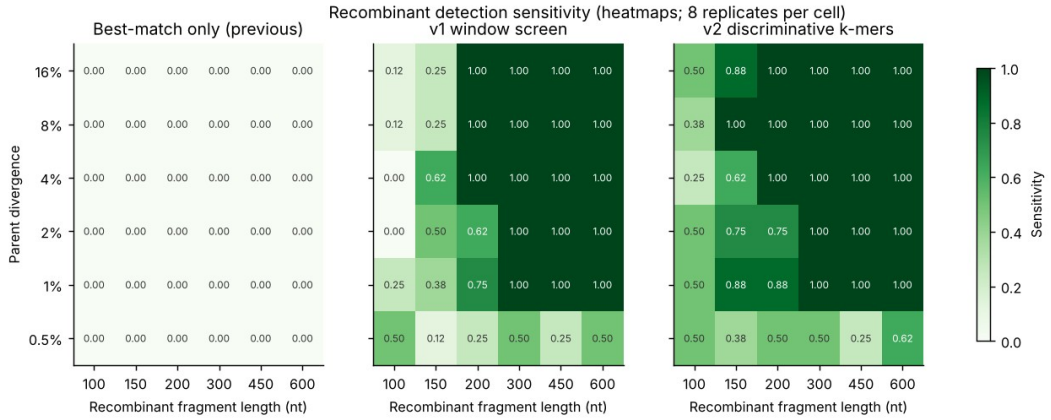


Figure 40. Recombinant detection sensitivity by parent divergence and fragment length (heat-maps).

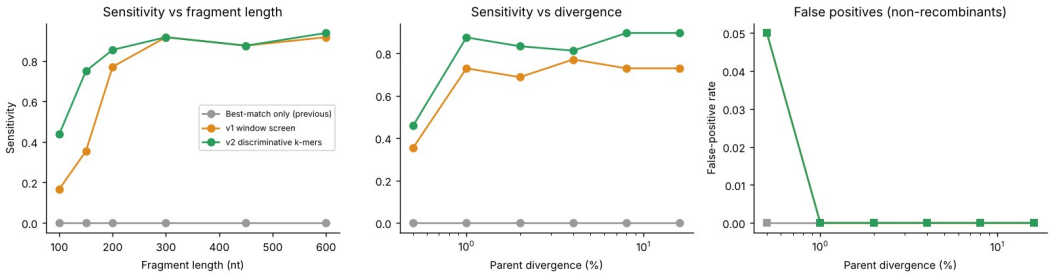


Figure 41. Sensitivity against fragment length and divergence, and false-positive rate.

7.3.9 Insecticide rotation

In a toy population-genetics model of resistance evolution (five mode-of-action groups, 60 sprays, 100 replicates, susceptible kill rate 0.8 and an assumed fitness cost of resistance), spraying one group repeatedly gave a mean efficacy of 0.04 (uniform initial resistance) and 0.03 (two groups already compromised), against 0.39 and 0.35 for the rotation planner (42, 43). The planners did **not** beat a fixed round-robin schedule (0.40 and 0.35), and every strategy lost efficacy within about 20 sprays. We read this as a lesson about the limits of rotation alone: with strong selection and few groups, efficacy cannot be preserved without reducing the number of sprays, for instance by thresholds, exclusion nets and resistant cultivars [6], [17]. The parameters of this model are illustrative and were not measured.

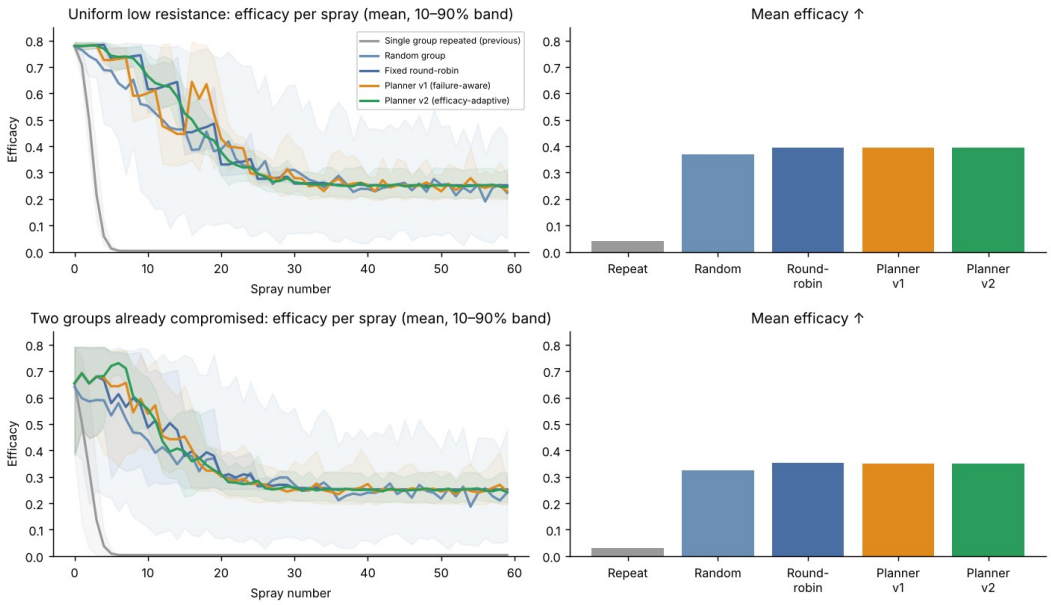


Figure 42. Efficacy of rotation strategies over 60 sprays in two scenarios.

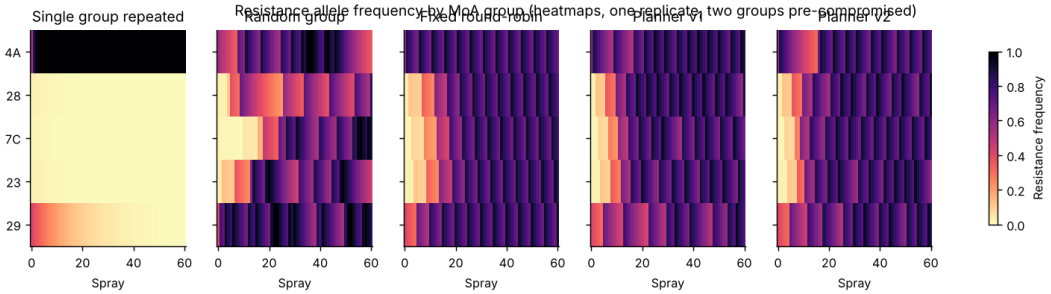


Figure 43. Resistance allele frequency by mode-of-action group for one replicate (heat-maps).

7.3.10 Seed-lot prevalence bound

The rule-of-three bound (44) is analytic: with *n* sampled seeds and no positives, the probability of seeing none when the true prevalence equals 3/*n* is (1 − 3/*n*)^n, which is about 5% for the sample sizes shown. The register therefore reports an upper 95% bound of 3/*n* for negative lots, and flags untested lots, because a negative sample cannot exclude low-prevalence infection when seed transmission is possible [3].

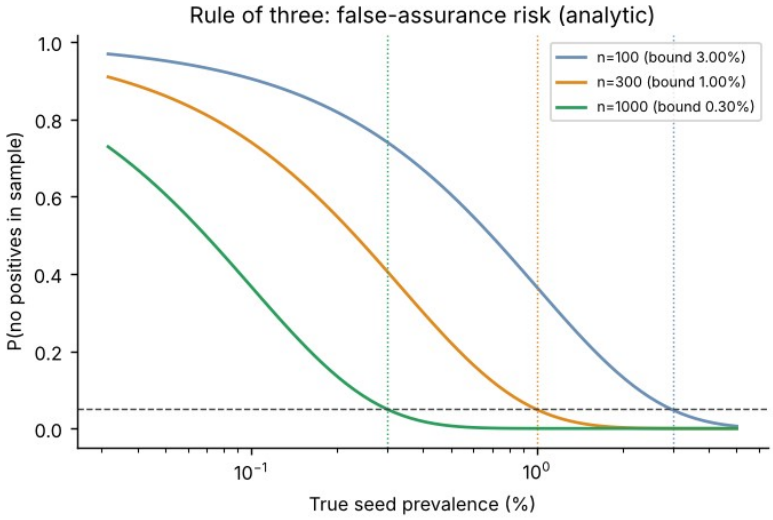


Figure 44. Probability of finding no positive seed against true prevalence for three sample sizes (analytic).

7.3.11 Overall comparison

45 summarises the simulated comparison on six axes: outbreak AUC, calibration, robustness to the unseen hot region, stacking rank correlation, recombinant sensitivity and resistance-management efficacy. The previous approaches are lowest on every axis, v1 intermediate and v2 highest on those where it differs, with the caveat that rotation efficacy is not improved by either planner relative to round-robin. The chart is a visual summary of the benchmark above and inherits all of its caveats.

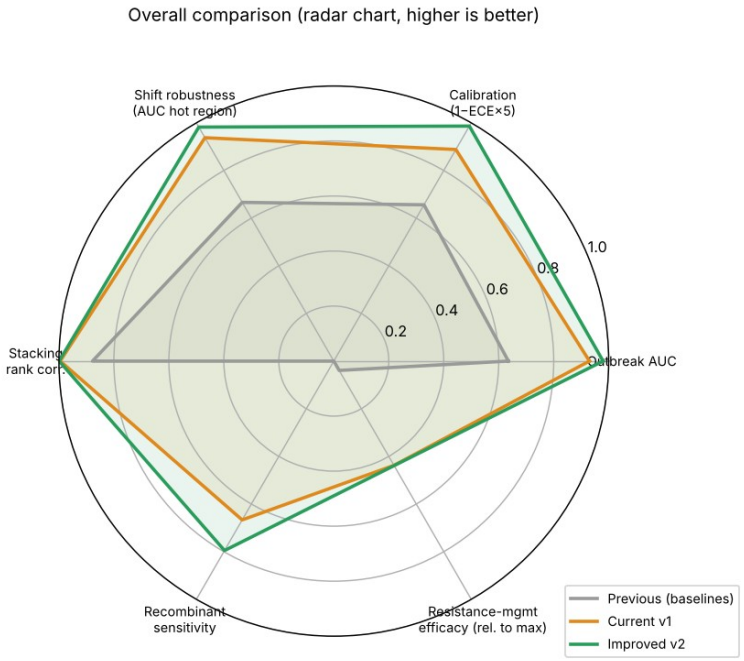


Figure 45. Overall comparison of previous approaches, v1 and v2 on the simulated benchmark (radar chart).

7.4 Software verification (RQ5)

All 34 automated tests pass (10). The full benchmark (3,600 simulated fields, all module benchmarks and 16 figures) runs in about 12 seconds on a four-core CPU, the sensitivity analyses in about a minute and the leakage experiment with three CNN seeds in around ten minutes, so the study can be reproduced on modest hardware. During testing, one genuine defect was found and fixed: the first hash used for duplicate grouping merged unrelated images, which was detected because it left a split empty; the group splitter now warns when one group holds more than 20% of the images.

7.5 Summary of evidence

Table 17. Claims supported by the evidence in this paper

Claim	Evidence	Strength and caveat
The public TYLCV folder cannot estimate diagnostic accuracy	Audit of 100 real photographs	Strong; descriptive, one folder
Lab-style and field photographs differ strongly in simple statistics	Audit and separability analysis	Strong for this folder; only 9 non-lab images
Random splits inflate accuracy when near-duplicates exist	Leakage experiment, 20 + 3 seeds	Mechanism demonstrated on synthetic images; magnitude not transferable
Calibrated model beats heuristic and single-factor baselines	Simulated benchmark with bootstrap CIs	Holds in simulation; shares assumptions with the simulator
Rotation planners improve on repeating one group	Toy resistance model	Yes, but not over fixed round-robin; illustrative parameters
The framework can be implemented and tested	34 tests, reproducible scripts	Strong for software; says nothing about accuracy
Field diagnostic accuracy of a deep model for TYLCV	None	Not established

8. Discussion

8.1 What the results do and do not show

The evidence in this paper is of three different kinds, and it supports three different kinds of statement. The audit is real evidence about a real dataset: the public TYLCV folder we examined has no negatives, is mostly lab-style, contains duplicates and differs strongly between lab-style and field photographs in simple statistics. The leakage experiment is a demonstration of a mechanism on synthetic images: a random split rewards memorisation of copies, and a duplicate-aware split removes most of that reward. The benchmark is simulation: it shows how the decision-support components behave when the world follows the simulator's rules. None of these is evidence that a deep-learning model can diagnose TYLCV in a real field with a stated sensitivity and specificity, and the paper does not claim that. The contribution lies in the framework, the tooling and the specification of the evidence still needed.

8.2 Why data and evaluation, rather than architecture, are the bottleneck

The literature reports accuracies above 99% on lab-background benchmarks [7], [8] and similar results across architectures [20], which means that benchmark accuracy no longer discriminates between methods. The audit shows concretely why such benchmarks are an unreliable guide to the field. A single statistic, the fraction of the frame covered by the leaf, separates lab-style from phone photographs without error, so any model trained on a mixture of such images can reach high accuracy by using the composition of the frame rather than the symptoms [12], [13]. The leakage experiment adds a second mechanism: when near-duplicates cross the train–test boundary, even a trivial learner such as nearest-neighbour on pixels looks accurate. A study that reports a single accuracy on a random split of a mixed dataset cannot exclude either mechanism.

8.3 Implications for collecting TYLCV image data

The findings translate into concrete requirements for a TYLCV image collection (18). The most important are a negative class that includes look-alike disorders, a mixture of capture conditions that is balanced across classes (so that composition does not predict the label), plant and session identifiers that allow grouping, and a molecular subset that anchors labels. Capture should follow a written protocol: photograph the youngest symptomatic leaves, include both a close-up and a plant-level view, record device, date, cultivar, growth stage and coarse location, and avoid reusing the same photograph under different names.

Table 18. Requirements for a TYLCV leaf-image collection suitable for evaluation

Requirement	Reason	Check
Healthy and look-alike negatives	Specificity cannot be measured without them	Per-condition counts
Capture conditions balanced across classes	Prevents composition shortcuts	Leaf coverage, background and device distributions by class
Plant and session identifiers	Allows group-aware splitting	Group size histogram; leak fraction
External sites and devices	Measures robustness to shift	Held-out site list
Molecular confirmation of a subset	Anchors labels to infection state	Fraction PCR-confirmed; agreement with visual labels
Metadata (cultivar, stage, date)	Explains failures	Completeness report
Licence and consent	Allows sharing and reuse	Licence recorded; farm consent for geo-tags

8.4 Interpreting a screening result in context

A screening result is a likelihood, not a verdict, and its meaning depends on the prior. 46 shows the positive predictive value of three hypothetical screening models as a function of prevalence (Equation 7); the models are not claimed to exist and are used only to show the effect. Even a model with 99% sensitivity and 95% specificity yields a PPV below 50% when fewer than 5% of plants are infected, and a model with 90% sensitivity and 90% specificity needs a prevalence of about 10% to reach a PPV of 50%. The practical consequence is that the same photograph

should lead to different actions early and late in an outbreak, which is why TYLCV-Guard presents image screening next to the risk index, vector information and seed-lot status rather than as a stand-alone verdict.

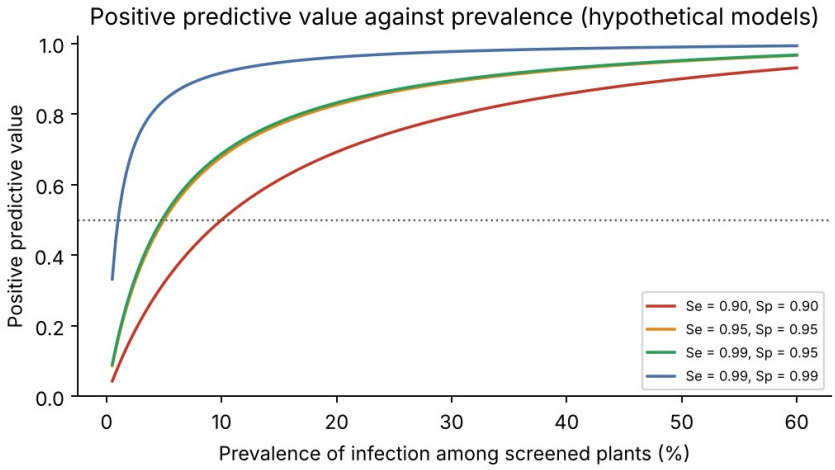


Figure 46. Positive predictive value as a function of prevalence for four hypothetical screening models (Equation 7).

8.5 Relation to earlier work

Our audit is consistent with earlier observations that lab-background benchmarks differ from field conditions [7], [14], [15] and that backgrounds can carry class information [12]; it adds a TYLCV-specific example with real photographs and a measurable statistic. The leakage experiment echoes general warnings about shortcut learning [13] and distribution shift [31] in a setting that can be reproduced in a few minutes. The simulated benchmark follows the common finding that calibrated, context-aware models outperform single-factor rules, but it cannot confirm it for real fields. The decision-support modules follow integrated management practice [6], and are limited, as explained, by assumed parameters.

8.6 Recommendations

- Report the leak fraction, the data sources and the label coverage of every image dataset used for TYLCV, and use group-aware splits.
- Evaluate on an external field set and on look-alike negatives before making claims about field use, and say which level of the maturity ladder has been reached.
- Report sensitivity and specificity with confidence intervals, calibrated probabilities and the threshold-selection rule, not accuracy alone.
- Check for shortcuts with a background-only baseline and with Grad-CAM, and treat saliency as a diagnostic aid [40].
- Present screening results with prevalence and vector context, and as a prompt for molecular confirmation.

9. Limitations and Threats to Validity

The most important limitation is that **no image classifier was trained or evaluated on real labelled leaf photographs**. The only real TYLCV photographs available to us were all positive; we could not obtain a labelled collection with healthy and look-alike negatives or ImageNet-pretrained weights in our environment; and a hosted image service was not reachable. The deep-learning pipeline is therefore verified for correctness on synthetic data but not for diagnostic accuracy. Further limitations follow.

- **Internal validity.** The simulator and the models share qualitative assumptions and were designed by the same authors, so the benchmark shows consistency with those assumptions, not their truth. The heuristic weights, the stacking weights, the rotation parameters and the rotation model are illustrative priors. The calibrated model was evaluated only on simulated data.
- **Construct validity.** An outbreak defined as 30% incidence is a convenient simulation label. Leaf coverage, yellowness and sharpness are crude statistics that do not measure symptom severity; the lower yellowness of field photographs mainly reflects the composition of the frame.
- **External validity.** The audit concerns one public folder of 100 images, of which nine are non-lab and seven independent. The leakage experiment uses synthetic images with a deliberately weak signal; its magnitude does not transfer to real data. The recombinant screen was tested on random sequences with independent substitutions, which is easier than real genomes.
- **Residual leakage.** The duplicate-aware split in the leakage experiment still had a leak fraction of about 19.2%, because some synthetic variants differed by more than the hash threshold. Hash-based grouping cannot detect different photographs of the same plant, which require metadata.
- **Conclusion validity.** The leakage experiment has three CNN seeds, so its standard deviation is itself uncertain; the classical learners have 20 seeds. Bootstrap intervals describe test-set sampling only.
- **Data licences.** The licence of the public image folder was not verified, so photographs should not be redistributed. The hosted Roboflow service was tested only with a mocked client.
- **Software.** The prototype has no authentication and is meant for local or trusted use; the web interface was tested with a browser automation script on demonstration data, not by users.

10. Ethical, Privacy and Safety Considerations

A screening tool that influences decisions about removing plants or spraying carries responsibilities. *Advisory status.* All outputs are labelled as advisory and image results as screening flags that need PCR or qPCR confirmation; the rotation planner gives no product or dose advice, because these depend on local registration and labels. *Privacy.* Geo-tagged farm images can reveal field locations and farm practices; the prototype stores only coarse country-level information in surveillance summaries, and a deployment should obtain consent and minimise location data. *Credentials.* API keys are read only from the environment, never stored in the code or the database, and error messages are sanitised. *Equity.* Tools should be tested with smallholders as well as commercial farms and should degrade gracefully with poor connectivity. *Environmental effects.* False alarms can lead to unnecessary spraying or removal, which has costs for growers and the environment, which is one reason for explicit threshold selection by error cost. *Intellectual property.* Public datasets should be used within their licences, and photographs in publications should be cleared for reuse.

11. Conclusion and Future Work

This paper presented a leakage-aware framework and a decision-support prototype for leaf-image screening of tomato yellow leaf curl virus, together with real and simulated evidence about the data and methods. An audit of a public TYLCV image folder showed that it is all-positive, mostly lab-style and contains duplicates, and that leaf coverage alone separates lab-style from field photographs. A controlled experiment showed that random splits inflate accuracy when near-duplicates exist and that duplicate-aware splitting removes most of the inflation. A simulated benchmark showed that a calibrated model improved on heuristic and single-factor baselines, with uncertainty quantified, and that rotation alone does not preserve insecticide efficacy. The prototype is implemented with the Python standard library, passes 34 automated tests and can be reproduced on modest hardware.

The central caveat is that no image classifier has yet been evaluated on real labelled leaf photographs. The next steps follow from the maturity ladder: (1) assemble a labelled collection with healthy and look-alike negatives, field photographs from several sites and devices, grouping metadata and a PCR-confirmed subset; (2) train the baselines and the transfer-learning classifier with several seeds under duplicate-aware splits and evaluate them on internal, external and hard-negative sets; (3) run shortcut checks with a background-only baseline and Grad-CAM, calibrate probabilities and fix thresholds by error cost; (4) compare the local model with a hosted workflow on the same test images; (5) test the integrated tool with growers, and (6) extend the decision-support layer with weather-data ingestion, calibration from real trials and field user studies. Only after these steps can the field diagnostic accuracy of deep learning for TYLCV be stated with confidence.

Acknowledgements

The authors thank the Department of Computer Science and Engineering, Chandigarh University, for the academic environment in which this work was carried out.

Declarations

Conflict of interest. The authors declare no conflict of interest. **Funding.** No specific funding was received for this work. **Data and code availability.** The code, benchmark scripts, figures and result files are in the project repository (TYLCV-Guard). The real photographs analysed are from a public Kaggle folder [50], which is not redistributed here; its licence should be checked before reuse. All simulated and synthetic data are generated by scripts with fixed seeds.

Appendix A: API Examples

Requests and responses are JSON. The examples below use the default local server (`python -m app.server --port 8080`).

Risk index. Request and (abridged) response:

```
POST /api/risk
{"temp_c": 28, "whitefly_per_leaf": 3, "stage": "vegetative", "protection":
0.45, "exclusion": false}

200 OK
{"index": 31.8, "category": "Moderate", "components": {"temperature": 1.0,
"vector": 0.77, ...}, "actions": [...]}
```

For these inputs the module returns an index of 42.4 (High).

Stacking.

```
POST /api/stacking
{"selected": ["Ty-1/Ty-3", "Ty-2"], "mean_max_temp_c": 33,
"recombinant_pressure": true, "virus": "TYLCV"}

200 OK
{"protection": ..., "durability": "Moderate", "recommendations": [{"locus":
"ty-5", "gain": ...}, ...], "warnings": [...]}
```

For these inputs the planner returns a protection score of 0.5134, durability 'Moderate' and the warnings: Ty-1/Ty-3 effect reduced for hot-season production.; Recombinant pressure: do not rely on Ty-1/Ty-3 alone..

Rotation.

```
POST /api/rotation
{"available_groups": ["4A", "28", "7C", "23", "29"], "history": ["4A"],
"sprays": 6, "failed_groups": [], "efficacy": {"28": 0.7}}

200 OK
```

```
{"plan": [{"spray": 1, "moa_group": ...}, ...], "usage": {...}, "warnings":
[...]}
```

For these inputs the planner proposes the sequence 7C, 23, 29, 28, 4A, 7C.

Leaf scan.

```
POST /api/diagnose
{"image": "...", "threshold": 0.5}

200 OK
{"verdict": "...", "tylcv_confidence": 0.0, "detections": [...], "advice":
[...]}
400 Bad Request {"error": "vision not configured; set
ROBOFLOW_API_KEY, ..."}
```

Appendix B: Configuration and Parameters

Table 19. Environment variables

Variable	Meaning
TYLCV_DB	Path of the SQLite database (default data/tylcv.db)
TYLCV_MODEL	Path of the calibrated model file (default data/model_v2.json)
TYLCV_LOCAL_MODEL	Path of a locally trained checkpoint; when set, Leaf scan uses it
ROBOFLOW_API_KEY, ROBOFLOW_WORKSPACE, ROBOFLOW_WORKFLOW_ID	Credentials and identifiers for the hosted workflow (never stored in code)
ROBOFLOW_API_URL, ROBOFLOW_CLASSES	Optional endpoint and class name

Table 20. Benchmark and simulator parameters

Parameter	Value
Regions and mean temperatures	cool 22 °C, warm 27 °C, hot 32 °C (standard deviation 3 °C)
Plants sampled per field	100 (varied 10–200 in the noise analysis)
Outbreak threshold	incidence ≥ 0.30
Train / test fields	2,400 / 1,200 (mixed regions); 2,400 cool+warm / 1,200 hot (shift test)
Recombinant pressure	probability 0.25 (cool, warm) or 0.45 (hot)
Rotation model	5 groups, 60 sprays, 100 replicates, kill rate 0.8, reversion 0.93 per spray for unsprayed groups
Recombinant test	divergence 0.5–16%, fragments 100–600 nt, query noise 1.5%, 8 replicates per cell
Random seeds	2026 (main benchmark) and fixed seeds listed in the scripts

Appendix C: Reproduction Instructions

From the `tylcv-guard` directory of the repository:

```
python -m unittest discover -s tests -t .          # 34 automated tests
python -m app.server --port 8080                    # web application
python benchmark/run.py                             # simulated benchmark,
15 figures
```

```
python -m experiments.sensitivity                    # bootstrap, noise,
training size, thresholds
python experiments/leakage_demo.py                   # synthetic leakage
experiment
python experiments/leakage_plot.py                   # leakage figure
python -I experiments/data_analysis.py <dir> <out>  # audit of an image
folder
python experiments/diagrams.py                       # schematic diagrams
python -m dl.train --data <dir> --out runs/exp1     # train and evaluate
(needs labelled negatives)
```

The benchmark and experiments need NumPy, Matplotlib, scikit-learn and (for the CNN) PyTorch; the application itself needs only the Python standard library.

Appendix D: Non-lab Photographs in the Audited Folder

Table 21. The nine non-lab photographs and their image statistics

File	Pixels	Leaf coverage	Sharpness	Yellowness
1723454502799.jpg	960×1280	0.049	4089	0.138
1723454502881.jpg	960×1280	0.016	4038	0.148
IMG20240812111153.jpg	3000×4000	0.013	3540	0.117
IMG20240812111244.jpg	3000×4000	0.006	4082	0.157
IMG20240812111249.jpg	3000×4000	0.025	4134	0.134
IMG20240812111314.jpg	3000×4000	0.021	3803	0.037
IMG20240812111320.jpg	3000×4000	0.007	3518	0.162
IMG20240812111527.jpg	3000×4000	0.093	7041	0.082
IMG20240812112103.jpg	3000×4000	0.117	9783	0.106

Pairs of photographs at hash distance below 12 bits are the same image at two resolutions: 1723454502799.jpg and IMG20240812111249.jpg (7 bits); 1723454502881.jpg and IMG20240812111244.jpg (1 bits).

References

[1] A. Prasad, N. Sharma, G. Hari-Gowthem, M. Muthamilarasan, and M. Prasad, Tomato yellow leaf curl virus: Impact, challenges, and management, *Trends Plant Sci.* 25(9) (2020) 897–911.

[2] H. Czosnek, A. Hariton-Shalev, I. Sobol, R. Gorovits, and M. Ghanim, The incredible journey of begomoviruses in their whitefly vector, *Viruses* 9(10) (2017) 273.

[3] E.-J. Kil *et al.*, Tomato yellow leaf curl virus (TYLCV-IL): A seed-transmissible geminivirus in tomatoes, *Sci. Rep.* 6 (2016) 19013.

[4] B. Mabvakure *et al.*, Ongoing geographical spread of *Tomato yellow leaf curl virus*, *Virology* 498 (2016) 257–264.

[5] Z. Yan *et al.*, Resistance to tomato yellow leaf curl virus in tomato germplasm, *Front. Plant Sci.* 9 (2018) 1198.

[6] M. R. Rojas *et al.*, World management of geminiviruses, *Annu. Rev. Phytopathol.* 56 (2018) 637–677.

[7] S. P. Mohanty, D. P. Hughes, and M. Salathé, Using deep learning for image-based plant disease detection, *Front. Plant Sci.* 7 (2016) 1419.

[8] K. P. Ferentinos, Deep learning models for plant disease detection and diagnosis, *Comput. Electron. Agric.* 145 (2018) 311–318.

[9] A. Kamilaris and F. X. Prenafeta-Boldú, Deep learning in agriculture: A survey, *Comput. Electron. Agric.* 147 (2018) 70–90.

[10] A. Abade, P. A. Ferreira, and F. de Barros Vidal, Plant diseases recognition on images using convolutional neural networks: A systematic review, *Comput. Electron. Agric.* 185 (2021) 106125.

[11] M. Shoaib *et al.*, An advanced deep learning models-based plant disease detection: A review of recent research, *Front. Plant Sci.* 14 (2023) 1158933.

[12] M. A. Noyan, Uncovering bias in the PlantVillage dataset, *arXiv:2206.04374*, 2022.

[13] R. Geirhos *et al.*, Shortcut learning in deep neural networks, *Nat. Mach. Intell.* 2 (2020) 665–673.

[14] J. G. A. Barbedo, Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification, *Comput. Electron. Agric.* 153 (2018) 46–53.

[15] M. Arsenovic, M. Karanovic, S. Sladojevic, A. Anderla, and D. Stefanovic, Solving current limitations of deep learning based approaches for plant disease detection, *Symmetry* 11(7) (2019) 939.

[16] S. H. Lee, H. Goëau, P. Bonnet, and A. Joly, New perspectives on plant disease characterization based on deep learning, *Comput. Electron. Agric.* 170 (2020) 105220.

[17] A. R. Horowitz, M. Ghanim, E. Roditakis, R. Nauen, and I. Ishaaya, Insecticide resistance and its management in *Bemisia tabaci* species, *J. Pest Sci.* 93 (2020) 893–910.

[18] H. Yamaguchi *et al.*, An NB-LRR gene, *TYNBS1*, is responsible for resistance mediated by the *Ty-2* begomovirus resistance locus of tomato, *Theor. Appl. Genet.* 131 (2018) 1345–1362.

[19] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, Deep neural networks based recognition of plant diseases by leaf image classification, *Comput. Intell. Neurosci.* 2016 (2016) 3289801.

[20] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, A comparative study of fine-tuning deep learning models for plant disease identification, *Comput. Electron. Agric.* 161 (2019) 272–279.

[21] M. Brahimi, K. Boukhalfa, and A. Moussaoui, Deep learning for tomato diseases: Classification and symptoms visualization, *Appl. Artif. Intell.* 31(4) (2017) 299–315.

[22] M. Agarwal, A. Singh, S. Arjaria, A. Sinha, and S. Gupta, ToLeD: Tomato leaf disease detection using convolution neural network, *Procedia Comput. Sci.* 167 (2020) 293–301.

[23] R. Karthik, M. Hariharan, S. Anand, P. Mathikshara, A. Johnson, and R. Menaka, Attention embedded residual CNN for disease detection in tomato leaves, *Appl. Soft Comput.* 86 (2020) 105933.

[24] S. Zhao, Y. Peng, J. Liu, and S. Wu, Tomato leaf disease diagnosis based on improved convolution neural network by attention module, *Agriculture* 11(7) (2021) 651.

[25] G. Geetharamani and A. Pandian, Identification of plant leaf diseases using a nine-layer deep convolutional neural network, *Comput. Electr. Eng.* 76 (2019) 323–338.

[26] J. Boulent, S. Foucher, J. Théau, and P.-L. St-Charles, Convolutional neural networks for the automatic identification of plant diseases, *Front. Plant Sci.* 10 (2019) 941.

[27] J. G. A. Barbedo, Plant disease identification from individual lesions and spots using deep learning, *Biosyst. Eng.* 180 (2019) 96–107.

[28] A. Picon *et al.*, Deep convolutional neural networks for mobile capture device-based crop disease classification in the wild, *Comput. Electron. Agric.* 161 (2019) 280–290.

[29] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, Deep learning for image-based cassava disease detection, *Front. Plant Sci.* 8 (2017) 1852.

[30] D. Singh *et al.*, PlantDoc: A dataset for visual plant disease detection, in: *Proc. ACM India Joint Int. Conf. Data Sci. Manage. Data (CoDS-COMAD)*, 2020, pp. 249–253.

[31] P. W. Koh *et al.*, WILDS: A benchmark of in-the-wild distribution shifts, in: *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 5637–5664.

[32] A. Fuentes, S. Yoon, S. C. Kim, and D. S. Park, A robust deep-learning-based detector for real-time tomato plant diseases and pests recognition, *Sensors* 17(9) (2017) 2022.

[33] P. Wspanialy and M. Moussa, A detection and severity estimation system for generic diseases of tomato greenhouse plants, *Comput. Electron. Agric.* 178 (2020) 105701.

[34] A. Howard *et al.*, Searching for MobileNetV3, in: *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314–1324.

[35] A. Ahmad, D. Saraswat, and A. El Gamal, A survey on using deep learning techniques for plant disease diagnosis and recommendations for development of appropriate tools, *Smart Agric. Technol.* 3 (2023) 100083.

[36] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, On calibration of modern neural networks, in: *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[37] B. Lakshminarayanan, A. Pritzel, and C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, in: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 6402–6413.

[38] Y. Ovadia *et al.*, Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift, in: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 13991–14002.

[39] R. R. Selvaraju *et al.*, Grad-CAM: Visual explanations from deep networks via gradient-based localization, in: *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[40] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, Sanity checks for saliency maps, in: *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018, pp. 9505–9515.

[41] U. Gill *et al.*, *Ty-6*, a major begomovirus resistance gene on chromosome 10, is effective against *Tomato yellow leaf curl virus* and *Tomato mottle virus*, *Theor. Appl. Genet.* 132 (2019) 1543–1554.

[42] E. Fiallo-Olivé and J. Navas-Castillo, Begomoviruses: What is the secret(s) of their success? *Trends Plant Sci.* 28 (2023) 715–727.

[43] E. Moriones, S. Praveen, and S. Chakraborty, Tomato leaf curl New Delhi virus: An emerging virus complex threatening vegetable and fiber crops, *Viruses* 9(10) (2017) 264.

[44] R. A. C. Dângelo, M. Michereff-Filho, M. R. Campos, P. S. da Silva, and R. N. C. Guedes, Insecticide resistance and control failure likelihood of the whitefly *Bemisia tabaci* (MEAM1; B biotype): A Neotropical scenario, *Ann. Appl. Biol.* 172(1) (2018) 88–99.

[45] S. Skendžić, M. Zovko, I. P. Živković, V. Lešić, and D. Lemić, The impact of climate change on agricultural insect pests, *Insects* 12(5) (2021) 440.

[46] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, in: *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[47] M. Tan and Q. V. Le, EfficientNet: Rethinking model scaling for convolutional neural networks, in: *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[48] A. Dosovitskiy *et al.*, An image is worth 16×16 words: Transformers for image recognition at scale, in: *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[49] Thesis, Tomato - Yellow Leaf Curl Virus Dataset, Roboflow Universe, Roboflow, Dec. 2022. [Online]. Available: https://universe.roboflow.com/thesis-okplj/tomato-yellow-leaf-curl-virus

[50] mohammeda5, Tomato Yellow Leaf Curl Virus Dataset, Kaggle. [Online]. Available: https://www.kaggle.com/datasets/mohammeda5/tomato-yellow-leaf-curl-virus-dataset